TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MAJOR PROJECT REPORT

ON

CIRCUITRON: CIRCUIT RECOGNITION, TRANSFORMATION, AND ANALYSIS

Submitted By:

Anveshan Timsina	(THA078BEI005)
Sandesh Dhital	(THA078BEI034)
Saroj Nagarkoti	(THA078BEI039)
Utsab Dahal	(THA078BEI046)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

March 2026

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MAJOR PROJECT REPORT

ON

CIRCUITRON: CIRCUIT RECOGNITION, TRANSFORMATION, AND ANALYSIS

Submitted By:

Anveshan Timsina (THA078BEI005)

Sandesh Dhital (THA078BEI034)

Saroj Nagarkoti (THA078BEI039)

Utsab Dahal (THA078BEI046)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Er. Kobid Karkee

March 2026

DECLARATION

We hereby declare that the project entitled, **"CIRCUITRON: Circuit Recognition, Transformation, and Analysis"** in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering **Electronics, Communication and Information Engineering** is a bona fide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

Anveshan Timsina (THA078BEI005)
Sandesh Dhital (THA078BEI034)
Saroj Nagarkoti (THA078BEI039)
Utsab Dahal (THA078BEI046)

Date: March 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a major project work entitled **"CIRCUITRON: Circuit Recognition, Transformation, and Analysis"** submitted by **Anveshan Timsina, Sandesh Dhital, Saroj Nagarkoti , Utsab Dahal** in partial fulfillment for the award of Bachelor of Engineering in Electronics, Communication and Information Engineering. The project was carried out under the special supervision and within the time prescribed by the syllabus.

We found that the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor's Degree of Electronics, Communication, and Information Engineering.

Project Supervisor
Er. Kobid Karkee
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

Head of Department
Er. Umesh Kanta Ghimire
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

March 2026

COPYRIGHT

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGEMENT

First, we would like to thank the Department of Electronics and Computer Engineering for providing us with an opportunity to learn and research on this project. We appreciate the department's assistance and recommendations in the selection and hopefully the successful completion of this project.

Special thanks to our supervisor, **Er. Kobid Karkee**, for his guidance, invaluable feedback, and constant support in regards to the project. His expertise and encouragement have been pivotal in shaping the direction, scope and implementation of this project.

Additionally, we would like to thank all of our teachers, friends, and other direct as well as indirect contributors for their indispensable support and encouragement during our study and execution of the project. We are also grateful to the authors of various papers that we have referenced for our project.

Anveshan Timsina (THA078BEI005)
Sandesh Dhital (THA078BEI034)
Saroj Nagarkoti (THA078BEI039)
Utsab Dahal (THA078BEI046)

ABSTRACT

CIRCUITRON is an integrated system designed to bridge the gap between traditional hand-drawn electrical schematics and modern circuit simulation tools. The project pipeline automates the transformation of handwritten circuit diagrams into CircuitJS-compatible text files which enables the users to interact with, simulate, and analyze circuits in an editable digital environment. Deep learning models such as YOLOv7 for component detection, two kinds of OCR models, TrOCR and also a custom OCR pipeline for text recognition, work together with BFS-based path finding wire detection algorithm to achieve the project's objectives of visualizing circuits as editable schematics, and allowing simulation. The system has achieved a mAP@0.5 of 95% for component detection, and a character accuracy of 84.5% for text recognition, and proper wire and node detection, for horizontal, vertical as well as diagonal wires, across varied drawing styles. A user-friendly interface allows manual corrections, schematic editing, and direct integration with CircuitJS for simulation. Additionally, the platform incorporates a DeepSeek v3.1-powered assistant to answer circuit-related queries, thus reinforcing learning through natural language interaction.

Keywords: circuit analysis, Multi-head BFS, simulation, TrOCR, YOLOv7.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AC	Alternating Current
AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BJT	Bipolar Junction Transistor
CER	Character Error Rate
CGHD	Circuit Graph Handwritten Diagrams
CNN	Convolutional Neural Network
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
DC	Direct Current
EDA	Electronic Design Automation
ELAN	Efficient Layer Aggregation Network
FET	Field-Effect Transistor
FSM	Finite State Machine
GNN	Graph Neural Network
GUI	Graphical User Interface
HSV	Hue, Saturation, Value
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IoU	Intersection over Union
JSON	JavaScript Object Notation
LLM	Large Language Model
LMDB	Lightning Memory-Mapped Database
LSTM	Long Short-Term Memory
LTspice	Linear Technology Simulation Program with Integrated Circuit Emphasis
mAP	Mean Average Precision
ML	Machine Learning
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
PAN	Path Aggregation Network
PCB	Printed Circuit Board
PHT	Probabilistic Hough Transform
Python	high-level general-purpose programming language
REST	Representational State Transfer

RF	Radio Frequency
RMS	Root Mean Square
SGD	Stochastic Gradient Descent
SPICE	Simulation Program with Integrated Circuit Emphasis
VQA	Visual Question Answering
YOLO	You Only Look Once

1 INTRODUCTION

"CIRCUITRON: Circuit Recognition, Transformation, and Analysis" is a system to convert hand-drawn electrical circuit diagrams into CircuitJS-compatible text files enabling simulation and graphical analysis along with digital visualization of the circuit in an editable schematic. The system also integrates a prompt-based Artificial Intelligence (AI) assistant (DeepSeek v3.1) to help answer queries related to the relevant circuit.

1.1 Background Information

The first step of learning about electrical and electronic circuits involves looking at and hand-drawing said circuits with the necessary components. These hand-drawn circuit diagrams continue to play an important role in electronics education and early-stage hardware design due to the convenience they offer. However, such hand-drawn diagrams prove to be limited in their utility in the current education landscape that prioritizes intuitive, visual understanding as they require significant mental gymnastics to facilitate any form of analysis of the circuits, more so when incorporating variation in parameters.

A slew of rather easy-to-use circuit simulation and analysis tools such as Linear Technology Simulation Program with Integrated Circuit Emphasis (LTspice) and KiCad exist, of course; and they continue to foster visual learning and analysis of circuits of a range of complexities. But these traditional Electronic Design Automation (EDA) tools lack the capability to accept hand-sketched circuit diagrams as input. Instead, users are required to redraw the circuits in software using Graphical User Interface (GUI) tools or by manually entering Simulation Program with Integrated Circuit Emphasis (SPICE) netlists; steps that are both tedious and error-prone [1, 2]. Be it due to the (admittedly not very steep yet inconvenient) learning curve of these tools or just the additional effort of having to redraw a circuit digitally (the dullness of which only goes up with circuit complexity), the instructors as well as the students often tend to neglect arguably essential simulation-based intuitive teaching and learning. This again encourages the same long-established rote learning practice instead of ushering in a more modern way of understanding circuits and electronics at large. It has been long proven that students tend to perform better when given the appropriate simulative manipulation and concept clarification tools, especially in the context of electrical and electronics circuits [3]. It then follows that the reluctance to use such tools is actively hindering the students' potential. Thus, the use of visual manipulative simulation tools is the need of the hour. In fact, it has been the need of the hour for quite a while. But, due to a lack of any uber-convenient method, it is yet to be popularly embraced, primarily in technologically nascent countries such as Nepal.

1.2 Motivation

In recent years, the mantra of the modern world has seemingly become “adapt or die”. The society regularly stomps on the tech-illiterate to make way for the ones willing to champion the use of new and shiny tools in all aspects of their lives. In an increasingly globalized economy, this is especially true for the up-and-coming students who need to compete with other students all over the world relying solely on their understanding of subject matter in their particular fields. In such case, it becomes essential that they are given the tools to enhance their learning and understanding even by minor increments. In the context of electronics, circuits are the fundamental building blocks that permeate each and every aspect of our “teched-out” world and it is important for learners in the field to have an intuitively in-depth understanding of this fundamental ingredient. Thus, there exists a motivation for a hassle-free method to analyze and understand circuits to deepen the necessary conception and comprehension.

The motivation stems not only from a deep-seated commitment to give the instructors and students tools to easily visualize and analyze circuits in the belief that it will genuinely improve the quality of professionals and engineers born out of universities, but also from the personal experience of the members involved in this project. As the electronics engineers of the near future who have had and will continue to have to deal with circuits in one form or another, the members involved in this project are uniquely positioned to understand the plight of a beginner in the study of electronics. And a tool which can convert a hand-drawn circuit into a digital schematic and also allow for analysis would go a long way in alleviating that plight.

1.3 Problem Statement

The EDA tools currently available do not support the direct transformation of hand-drawn circuit images into simulation-ready schematics. There exist a few nascent prototypes for the same but there seems to be no widely adopted, robust, and user-friendly pipeline that combines component recognition, reliable wire connectivity, schematic generation and simulation, whether by itself or by integration with SPICE simulators.

“CIRCUITRON: Circuit Recognition, Transformation, and Analysis” project aims to create an integrated system capable of first, recognizing and digitizing hand-drawn circuits into editable schematics and then allowing the user to analyze those circuits through graphs using CircuitJS-compatible text files with an added AI assistant to handle related queries.

1.4 Project Objectives

- To automatically convert hand-drawn circuit diagrams into CircuitJS-compatible text files for simulation.
- To visualize the circuit in an editable schematic in addition to an AI assistant for handling simple, relevant queries.

1.5 Project Scope and Applications

1.5.1 Project Scope

This project focuses on the development of a software pipeline that accepts scanned or photographed images of hand-drawn electrical circuits and transforms them into CircuitJS-compatible netlists for simulation as well as editable schematics through an intermediate integrated data structure to facilitate the transformation. The scope includes detecting common circuit components (14, to be exact, including supplementary components like gnd, crossover, etc.) and their values in mentioned units. It infers wire connectivity with junction recognition and generates a circuit diagram with simulation capabilities. Additionally, the system incorporates an Large Language Model (LLM)-based (Deepseek v3.1) query-solving assistant. The system excludes any form of advanced circuits such as those associated with Radio Frequency (RF) or mixed-signal, and Printed Circuit Board (PCB) designs.

1.5.2 Project Applications

- The system is foremost, an educational tool designed to assist students and instructors alike in conveniently digitizing circuit diagrams and understanding circuit behavior through limited simulation and interactive Q&A.
- The project enables quick feedback and validation of hand-drawn designs against simulation results early in the design process.
- The project can facilitate quick on-the-go analysis of circuits for students and potentially, field engineers as well.

1.5.3 Project Limitations

Despite its innovative concept, the system is constrained by several factors. First, its detection and recognition accuracy is heavily dependent on the clarity of input images. Low-res or messy sketches may lead to recognition errors. Moreover, the system supports only a fixed set of commonly used components. This makes it unable to handle rare components and limits its functionality. Wire connectivity mapping is achieved through a junction-based approach, which assumes standard drawing practices. However,

there is no one way to draw a circuit diagram; non-standard and overlapping wires, or complicated circuits may confuse the system. Additionally, the system lacks a simulation engine of its own and instead uses an open-source tool like CircuitJS. This limits the simulation capabilities to that of CircuitJS. More rigorous training on appropriate data would help remedy some of these issues, however finding the ideal dataset with enough handwritten examples for all kinds of components and circuits has proven challenging.

1.6 Report Organization

The report begins with an Introduction which includes background information, motivation, problem statement, project objectives, applications, and limitations. Literature Review presents a summary of existing relevant research and their corresponding limitations that are in turn addressed by this project. Following this, Requirement Analysis simply describes the project requirements i.e, software needs, on account of it being a software-only project, along with feasibility study (economic, time, operational, technical). System Architecture and Methodology explains the system design through diagrams like block diagram, flowchart, use-case diagram, and architecture diagrams for specific modules used in the project. These diagrams are supplemented with serviceable explanations for each element in the project pipeline. It also includes dataset analysis and performance metrics expectation tables. Implementation Details clarifies practical aspects of the system, starting with preprocessing techniques, and then the model training details. This is followed by relevant algorithms for wire detection and connectivity and error handling techniques. Results and Analysis detail project outcomes i.e, the results of object detection, text recognition and line detection, in addition to performance analysis of different models used (with the use of graphs). Future Enhancements propose potential improvements or extensions and the Conclusion presents a summary of the entire project from problem statement to results and recommendations for future work. Appendices contain budget breakdown and Gantt chart as supplementary material to the main content of the report.

2 LITERATURE REVIEW

The automatic recognition of hand-drawn circuit diagrams has been explored since the early 2000s. In one of the earliest studies, Edwards and Chandran [4] proposed an approach that first identified circuit components and then inferred node segments by considering spatial relationships between them. Their system achieved an accuracy of 86% for component recognition and 92.5% for node identification, with wire connections traced between identified nodes using hard-coded heuristic conditions. While pioneering for its time, this approach was inherently brittle due to its reliance on manually crafted rules and could not generalize to diverse handwriting styles or circuit topologies. Ramel et al. [5] addressed a related problem in the broader domain of technical document recognition, proposing a structured approach that separated graphical primitives from textual annotations followed by structural matching against predefined symbol templates. Belongie et al. [6] introduced shape context descriptors for object recognition, computing histograms of relative contour point positions to provide descriptors invariant to small deformations, a useful property for hand-drawn symbols. While these early methods laid the groundwork, they were limited by their inability to learn discriminative features from data, motivating the shift toward deep learning-based approaches.

The detection and classification of individual circuit components is a foundational step in any circuit recognition pipeline, and modern approaches rely heavily on deep learning-based object detection architectures. The R-Convolutional Neural Network (CNN) family of detectors introduced region proposals for object detection: Girshick et al. [7] proposed R-CNN using selective search for candidate regions, followed by Fast R-CNN [8] which shared convolutional features across proposals, and Faster R-CNN [9] which replaced selective search with a Region Proposal Network. He et al. [10] extended this with Mask R-CNN, adding instance segmentation capabilities. In the circuit domain, Bayer et al. [11] leveraged Mask R-CNN for instance segmentation of components in hand-drawn diagrams, achieving approximately 94% mask accuracy, although they noted that symbol recognition was "reasonable, yet incomplete," particularly in complex drawings with inconsistent handwriting. Single-shot detectors eliminate the region proposal stage for real-time inference; Liu et al. [12] introduced SSD, which discretizes the output space into default bounding boxes at multiple scales. The YOLO family has been especially influential: Redmon et al. [13] proposed the original YOLO architecture framing detection as a single regression problem, followed by YOLOv2 [14] with batch normalization and anchor boxes, YOLOv3 [15] with feature pyramid predictions at three scales, YOLOv4 [16] with Mish activation and Cross-Stage Partial connections, and YOLOv5 [17] which gained widespread adoption through its PyTorch implementation. Wang et al. [18] proposed YOLOv7, introducing the (E-Efficient Layer Aggregation Network (ELAN)) and model re-parameterization techniques that achieved

state-of-the-art results on MS COCO, outperforming all prior real-time detectors in both speed and accuracy. YOLOv7 is the architecture employed in the present project for component detection.

In a 2021 study, Adnan et al. [19] developed a two-stage CNN-based system for classifying 20 different hand-drawn circuit components, achieving a recognition accuracy of 97.33%. Reddy and Panicker [1] combined YOLOv5 for component detection with probabilistic Hough transform-based node recognition, achieving a mAP of 98.2% in component detection and an overall schematic generation accuracy of 80%, though their system did not handle labeled components. Abhinav et al. [20] proposed a two-step method of detecting and classifying components followed by feeding labeled component strings into an Finite State Machine (FSM) for circuit recognition.

Extracting component values such as "10kΩ" or "5V" from hand-drawn annotations is critical for generating functionally accurate netlists. Shi et al. [21] proposed the Convolutional Recurrent Neural Network (CRNN) architecture, integrating convolutional feature extraction, bidirectional Long Short-Term Memory (LSTM) sequence modeling, and Connectionist Temporal Classification (CTC)-based transcription into a unified framework that has become the de facto standard for scene text recognition and forms the basis of the OCR module in the present project. Graves et al. [22] originally introduced the CTC loss function for labeling unsegmented sequence data, with the key advantage that it does not require pre-segmented training data, which is critical for handwritten text where character boundaries are ambiguous. Baek et al. [23] conducted a comprehensive comparison of scene text recognition models, revealing that attention-based decoders generally outperform CTC-based decoders on irregular text while CTC remains competitive on regular text, informing the design decision in this project to adopt a CTC-based CRNN for circuit text which tends to be horizontally aligned. More recently, transformer-based architectures such as TrOCR [24] have demonstrated strong performance on handwritten text recognition by leveraging pre-trained vision transformers, though they require substantially more computational resources, making them less suitable for lightweight systems like the one proposed here.

Detecting wires and inferring electrical connectivity is arguably the most challenging aspect of circuit diagram recognition, as wire detection must contend with arbitrary orientations, intersections, crossovers, and junctions. The Hough transform, originally proposed by Duda and Hart [25], is a classical technique for detecting straight lines, and its probabilistic variant introduced by Matas et al. [26] operates more efficiently on random subsets of edge points. Reddy and Panicker [1] used the Probabilistic Hough Transform for wire detection in hand-drawn circuits but encountered difficulties with curved or overlapping wires. Morphological operations such as the parallel thinning

algorithm proposed by Zhang and Suen [27] have also been employed for wire extraction, reducing binary regions to single-pixel-wide skeletons while preserving topological connectivity. Hemker et al. [28] proposed a three-stream approach (one for component detection, one for line and junction detection, and one for text recognition) with outputs combined into a JavaScript Object Notation (JSON) representation from which a netlist is derived. Kelly and Cole [29] addressed wire detection in printed circuit schematics, achieving 86.4% digitization success and 99.6% OCR accuracy, though their system was trained exclusively on computer-generated diagrams and cannot handle hand-drawn variability. An alternative approach formulates wire tracing as a graph search problem where the binarized circuit image is treated as a grid graph and breadth-first search or A^* algorithms find paths between component terminals; the present project adopts this path-finding approach after initial experiments with Hough transform-based methods proved inadequate.

Representing circuits as graphs, where nodes correspond to components or connection points and edges represent electrical connections, enables topological reasoning and downstream simulation. Kipf and Welling [30] introduced Graph Convolutional Networks that generalize convolutions to graph-structured data. Yamakaji et al. [31] developed Circuit2Graph, transforming circuit diagrams into graph structures and achieving 97.9% accuracy on circuit classification tasks using Graph Neural Networks (GNNs), demonstrating that graph neural networks can effectively model the global structure of a circuit even when visual features alone are insufficient.

The ultimate goal of many circuit recognition systems is to produce a netlist that can drive simulation. Traditional EDA tools such as KiCad and LTspice support netlist export from manually created schematics but do not allow direct image-to-netlist conversion [32]. Sertdemir et al. [2] developed Img2Sim, an artificial neural network-based system that detects components, infers connectivity, and outputs a complete SPICE netlist with over 90% accuracy in topology extraction, though it requires clean input images and is limited to a predefined set of components. Nagel and Pederson [33] developed SPICE at the University of California, Berkeley, establishing the standard for circuit simulation that persists to this day, with modern derivatives including Ngspice for Direct Current (DC), Alternating Current (AC), and transient analysis, and commercial tools such as PSpice and Simulink. CircuitJS [34], a web-based interactive circuit simulator providing real-time visualization of voltage, current, and waveform behavior, is the simulation platform adopted by the present project due to its open format and accessibility.

The emergence of LLMs has opened new possibilities for circuit understanding and explanation. The CircuitVQA dataset contains more than 115,000 question-image pairs based on 5,725 unique circuits covering approximately 70 symbol types [35], supporting

transformer-based Visual Question Answering (VQA) models that outperform generic systems when fine-tuned on domain-specific data. Vungarala et al. [36] developed SPICEPilot, leveraging GPT-based models to generate and interpret SPICE code, though general-purpose models often fail to respect domain conventions, underscoring the need for domain-adapted models. The present project integrates DeepSeek as an LLM-based assistant for answering circuit-related queries. Chen et al. [3] studied the efficacy of simulation-based learning in electronics education, finding that interactive circuit simulations significantly improved student comprehension, motivating the design of CIRCUITRON as not merely a recognition tool but an educational platform.

While substantial progress has been made in component recognition, value extraction, connectivity inference, netlist generation, and interactive Q&A, existing systems remain fragmented or limited in scope. Most systems address only one or two stages of the recognition pipeline; several [28, 29] were designed for printed schematics and do not generalize to hand-drawn circuits; none generate an editable, interactive schematic; VQA and LLM-based tools have not been integrated into recognition pipelines; and many systems [1, 11] do not extract handwritten component values. CIRCUITRON addresses these gaps by providing a unified, end-to-end pipeline that takes a hand-drawn circuit image as input and produces an editable, simulation-ready digital schematic with integrated LLM-based assistance. Table 2-1 summarizes the capabilities of the reviewed systems.

Table 2-1 Comparison of Existing Circuit Recognition Systems

System	**Hand-drawn**	**Component Detection**	**Value Extraction**	**Wire Detection**	**Netlist/ Simulation**	**AI Assist**
Edwards [4]	Yes	Yes	No	Yes	No	No
Adnan et al. [19]	Yes	Yes	No	No	No	No
Kelly & Cole [29]	No	Yes	Yes	Yes	No	No
Reddy & Panicker [1]	Yes	Yes	No	Yes	Yes	No
Bayer et al. [11]	Yes	Yes	No	Yes	No	No
Hemker et al. [28]	No	Yes	Yes	Yes	Yes	No
Img2Sim [2]	Yes	Yes	No	Yes	Yes	No
Circuit2Graph [31]	No	Yes	No	No	No	No
SPICEPilot [36]	No	No	No	No	Yes	Yes
CIRCUITRON	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

3 REQUIREMENTS ANALYSIS

3.1 Software Requirements

3.1.1 Python Libraries

Python handles the machine learning side of the project along with portions of the backend. Its collection of packages and libraries made it convenient to train and implement models for object detection and text recognition alike.

- **PyTorch and TensorFlow:** PyTorch and TensorFlow together form the computational backbone of this project. YOLOv7 and OCR sit inside the pipeline through these two frameworks, and fine-tuning happens whenever the training data changes. PyTorch tends to be the pick during research phases because its dynamic computational graphs let the network structure be altered mid-run. TensorFlow, on the other hand, gets called upon in later stages since its production ecosystem is more mature. Both maintain a direct link to NumPy and Pandas, so data preprocessing stays smooth with no compatibility headaches.
- **Numpy:** Numpy is an open-source Python library built to handle numerical computing and data manipulation. CIRCUITRON uses it for array operations and pixel-level work on images. It couples well with OpenCV too, so image processing tasks like converting images into matrices and running image-wide or element-wise computations for augmentation, thresholding, and segmentation become straightforward.
- **OpenCV:** OpenCV handles most of the circuit image pre-processing and the lower-level computer vision work. Thresholding, edge detection, contour tracing, color space conversions; virtually every image processing step in the project touches OpenCV at some point. The visual information pulled out through these operations is what eventually gets passed into the detection models.
- **Pandas:** Pandas brings data frames and similar structures that make wrangling tabular data simple. Reading files, cleaning messy entries, filtering rows, grouping records; it covers all of that out of the box. Within this project, Pandas keeps the structured circuit-component data in order: component lists carrying attributes like label, value, position, and orientation are stored through it. It also manages the intermediate data that flows between the detection stage, parsing, and netlist generation, and it can export tables or logs whenever debugging or evaluation calls for them.
- **Matplotlib and Plotly:** Matplotlib is a popular Python plotting library, and in this project it renders the performance metric scores as graphs. It will also be tapped for plotting simulation outputs once CircuitJS integration comes into play. Plotly fills a complementary role by supporting interactive diagrams, which is particularly handy for displaying simulation graphs that move over time.

- **Scikit-learn:** Scikit-learn comes in during the post-processing stage. Clustering junction points, weeding out outliers, classifying component types that the detector found ambiguous; those are the sorts of jobs it takes on. Its built-in utilities for data splitting, metrics, and evaluation proved quite handy while developing and benchmarking the models. Where needed, it can also pitch in with component matching, value prediction, or running validation models.
- **Seaborn:** Seaborn sits on top of Matplotlib and makes statistical graphics look polished with very little tweaking. Circuitron uses it to plot distributions of component detection accuracy and OCR confidence scores. Being a thin wrapper around Matplotlib, it keeps the graphing code short and readable. Paired with Pandas, it can also chart trends from simulation results, like voltage traces across components over time.
- **Hugging Face:** Hugging Face provides a large hub of pretrained transformer models for Natural Language Processing (NLP), vision, and more. It is used in the project to integrate LLMs for answering circuit-related queries and working with TrOCR.

3.1.2 YOLOv7

The YOLOv7 model is used to detect and classify circuit components in hand-drawn circuit images. It outputs bounding boxes, class labels, and confidence scores for each detected component. After comparison with other version of YOLO (v5, v8, v11), YOLOv7 is selected due to its superior detection accuracy, efficient training capabilities, and support for multiple object scales through its enhanced architecture, while also being of an appropriate size in regards to speed and training time.

3.1.3 OCR models

A CRNN-based OCR model trained from scratch, following the DTRB design philosophy for recognizing text is vital for the project as it is the core technology which reads and understands the component values and units of circuit elements. This has been chosen for the project because, foremost, other ready-made OCR models proved to be either too resource-hungry and/or failed to perform well even after some fine-tuning. Additionally, a minimally fine-tuned TrOCR model has also been used. The uses essentially gets to choose between a faster but slightly inferior custom OCR model, or a Microsoft-built robust yet slower TrOCR.

3.1.4 KiCad Symbol Library

KiCad is a free, open-source EDA suite which offers its libraries to be used freely to build any educational project. The project uses the KiCad symbols library, specifically, in order to simply extract the component symbols standardized by KiCad rather than

building the icons/representations for components from scratch for use in the frontend schematic.

3.1.5 Coding and Training Platforms

- **Google Colab:** Google Colab is a cloud-based platform that offers a Jupyter notebook environment for developing deep learning systems. For this project, Google Colab provided with Graphics Processing Units (GPUs) for free, which significantly accelerated the training process, enabling experimentation with larger models and datasets. TensorFlow, PyTorch, and other common Machine Learning (ML) libraries come pre-installed, so YOLO (v5 and v7) along with OCR could be implemented and tested with minimal setup overhead.
- **Kaggle:** Kaggle is a multi-faceted online platform popular for data science competitions, datasets, code sharing and model training. It was primarily used to find relevant datasets (Circuit Graph Handwritten Diagrams (CGHD), VQA, Handwritten Circuit Diagram (HCD)), code sharing and model training. The dataset of choice for this project was also found in Kaggle. Additionally, it provided a fully hosted environment with popular Python libraries such as Pandas, Numpy, Tensorflow, PyTorch pre-installed to make implementing and training models convenient, especially with its integration with Google Colab. Its free cloud computing service providing 30 hours of GPU use per week proved indispensable in accelerating the training and testing of different models.
- **Lightning AI:** Lightning AI is a cloud workspace, similar to Google Colab and Kaggle, which specializes in helping build agents and train models without any tedious setup. Most importantly, through the use of a student account, it allows for the use of extremely powerful GPUs for free (for a reasonable amount of time). Among all the cloud computing services used for the project, Lightning AI proved the most invaluable for implementing and training the models as the GPUs it allotted were super-fast and saved a ton of time during the tedious and time-consuming process.
- **Visual Studio Code:** VS Code is the local Integrated Development Environment (IDE) everyone on the team settled on, mainly for its auto-completion, syntax highlighting, and copilot integration. Each member extends it further with their own set of extensions tailored to their workflow. All locally executed code runs through VS Code, and its usefulness increased ten-fold once the web development phase began.

3.1.6 Web Development

- **Reactjs:** Reactjs serves as an integral technology underlying the dynamic front-end interfaces to be developed for the project. The frontend for this project includes dynamic elements like dragging, erasing, resizing, etc. of the circuits and circuit

elements, and Reactjs fulfills these requirements with convenience and speed through the use of its features like virtual Document Object Model (DOM) abstraction, distinction of code into reusable logic and presentation buckets, etc. as well as compatible packages like react-draggable.

- **Next.js:** Next.js wraps around React and adds routing, server-side rendering, and built-in Application Programming Interface (API) hooks. Those capabilities made it a natural fit for building the client-side interface. In practice, it ties the frontend to the backend services and keeps the recognition results rendering smoothly.
- **FastAPI:** FastAPI is a Python web framework focusing on building Representational State Transfer (REST) APIs quickly without sacrificing speed. It auto-generates interactive API docs, which saved time during development. The backend services that field frontend requests, shuffle data around, and talk to the recognition and analysis modules all run through FastAPI. Its async support and tight coupling with Pydantic keep request handling snappy and the data well-validated.
- **Uvicorn:** Uvicorn is an Asynchronous Server Gateway Interface (ASGI) server built for running async Python web apps with low latency. It pairs naturally with FastAPI and handles concurrent requests well. In the project, Uvicorn hosts the backend API and keeps the communication between the frontend and the OCR modules fast.
- **Pydantic:** Pydantic uses Python type annotations to validate data and define schemas. Any data moving between the frontend and backend gets checked against predefined structures through it, which cuts down on runtime errors. Request and response models in this project are all defined with Pydantic.
- **Github:** Github provides version-controlled code storage and sharing. All of the project's code lives there, and it has kept every team member synchronized throughout development.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Technical Feasibility asks whether the project can actually be built with the resources and know-how at hand. The technologies involved, Python, OpenCV, custom OCR, YOLOv7, FastAPI, Next.js, are all mature and carry strong community backing. YOLOv7 handles component detection well enough for handwritten circuit symbols. Wire and node tracing relies on established methods like skeletonization, junction-based mapping, and clustering. Two OCR paths exist: TrOCR on one side and the custom-trained model on the other, which together hold up even when the text is noisy. CircuitJS takes care of simulation. The team itself is composed of electronics and software engineers who are comfortable with these tools. Low-quality or cluttered input images pose the biggest

technical risk, though manual correction features built into the system soften that concern. All things considered, the project clears the technical feasibility bar.

3.2.2 Economic Feasibility

Economic Feasibility looks at whether the project justifies its costs. Computation expenses (Google Colab Pro and cloud GPUs) come to NPR 24,000. An IEEE membership for accessing research journals adds NPR 2,160. Miscellaneous spending on printing, presentations, and other memberships rounds out at NPR 15,000. Data collection costs nothing because open-source datasets and tools cover that entirely. The total budget sits at NPR 41,160. Given that the tool has real potential as a product for academic institutions, commercialization is a plausible path forward. Economically, the project holds up.

3.2.3 Operational Feasibility

Operational Feasibility gauges how well the system would work in a real setting. There is a genuine gap it fills: converting hand-drawn circuits into simulations without forcing the user to redraw everything from scratch. The interface lets users tweak results manually if needed, which adds a layer of reliability. Being web-based (Next.js and FastAPI), it runs on any platform with a browser. The learning curve stays low because the design is kept intuitive, and processing speeds stay close to real-time thanks to YOLOv7 and the lean OCR pipeline. A modular architecture means maintenance is manageable and new features can be tacked on later. The project passes the operational feasibility check.

3.2.4 Time Feasibility

Development was split into two sequential phases. Phase A covered the core: the circuit recognition and text extraction engine plus netlist generation, all wrapped inside a basic web framework. That portion was finished and tested by September of this year.

Phase B broadened the scope. The full web application was built out, CircuitJS was wired in for simulations with an interface for viewing and editing schematic diagrams, and the AI question-answering feature was implemented. All of this stayed consistent with the wireframes designed beforehand. A two-week buffer sat between the phases for thorough testing and adjustments; Phase B work only kicked off after Phase A components had been fully validated. This phased approach kept the design intact while layering on the more advanced features over time.

As of yet, all of the promised features have been delivered with only minor kinks to be sorted out. The project is considered feasible from a time perspective.

4 SYSTEM ARCHITECTURE AND METHODOLOGY

4.1 Block Diagram

The process of converting a handwritten circuit diagram into a structured netlist involves a series of coordinated steps, each step feeding into the next, designed to detect, extract, and represent pictorial information in a machine-readable format and then using this data for simulation and schematic generation.

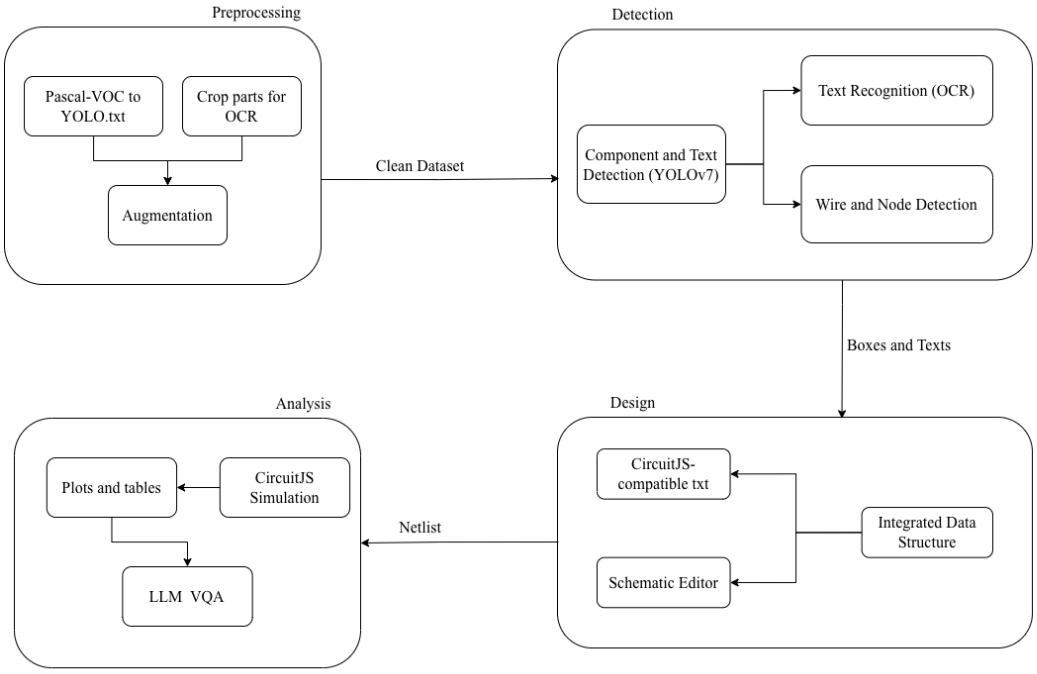


Figure 4-1 Block Diagram of the CIRCUITRON System

The high-level architecture of the CIRCUITRON system is visualized in the block diagram in Figure 4-1. Each step involved in the hand-drawn image to simulation-ready schematic pipeline is elaborated as follows.

4.1.1 Preprocessing

At first, the circuit information (components, component values, units, wire intersection, etc.) needs to be extracted out from the hand-drawn image. Appropriately fine-tuned ML and DL models are to be used for each of these steps which will be elaborated on in later subsections. However, for any sort of detection to happen, the image needs to first be pre-processed. Preprocessing for this not only includes image manipulation tasks but also conversion of the dataset annotations into appropriate formats to maintain compatibility with the component detection and text recognition models. The dataset of

choice is in the Pascal VOC format which is incompatible with the models used in the pipeline and hence need to be converted.

Furthermore, image preprocessing, which includes steps like thresholding, contrast and brightness enhancement (for improved clarity) is a vital step in readying the dataset for training any and all models. However, geometric augmentations, especially rotation, is not appropriate for this project.

Among the various available preprocessing techniques, the ones have been selected with careful consideration of the type of images available in the dataset as well as the models in use. Besides image manipulation, preprocessing also includes splitting the dataset into training, validation and testing subsets as appropriate.

4.1.2 Circuit Component Detection

A preliminary yet critical step in the project pipeline is the accurate detection of circuit components from hand-drawn schematic images. This is addressed using a real-time object detection algorithm which is capable of identifying and localizing multiple electronic components (such as resistors, diodes, etc.) directly within the image. YOLO algorithm, the YOLOv7 model, specifically, has been selected for the component detection needs of this project. It is to be noted that text, as a generic collection of letters, numbers, and special characters is also considered a component in this context.

YOLO (You Only Look Once) is a real-time object detection algorithm known for speed and efficiency. YOLO frames object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. This model works as follows: first, dividing the image into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose centre falls within it. For each cell, a number of bounding boxes and their corresponding confidence scores (which represents the accuracy of the box and whether the box contains an object) are predicted. Each predicted bounding box includes the coordinates of the box centre relative to grid cell, represented by (x, y), the width and height relative to the whole image, represented by (w, h) and the confidence score C given as:

$$C = P(\text{object}) \cdot \text{IoU} \tag{4-1}$$

where $P(\text{object})$ denotes the probability of an object being inside the box, and Intersection over Union (IoU) is the Intersection over Union between the predicted box and the ground truth box.

Additionally, for each box, class probabilities are predicted as:

$$\text{Class Score} = P(\text{class } i \mid \text{object}) \cdot C \tag{4-2}$$

This allows the model to classify which object exists in each box and once the predictions are made, Non-Maximum Suppression (NMS) is applied to discard less-relevant overlapping boxes.

Based on similar fundamental working principle, several iterations (versions) of YOLO have been introduced and popularized among which YOLOv7 is the YOLO model of choice for this project.

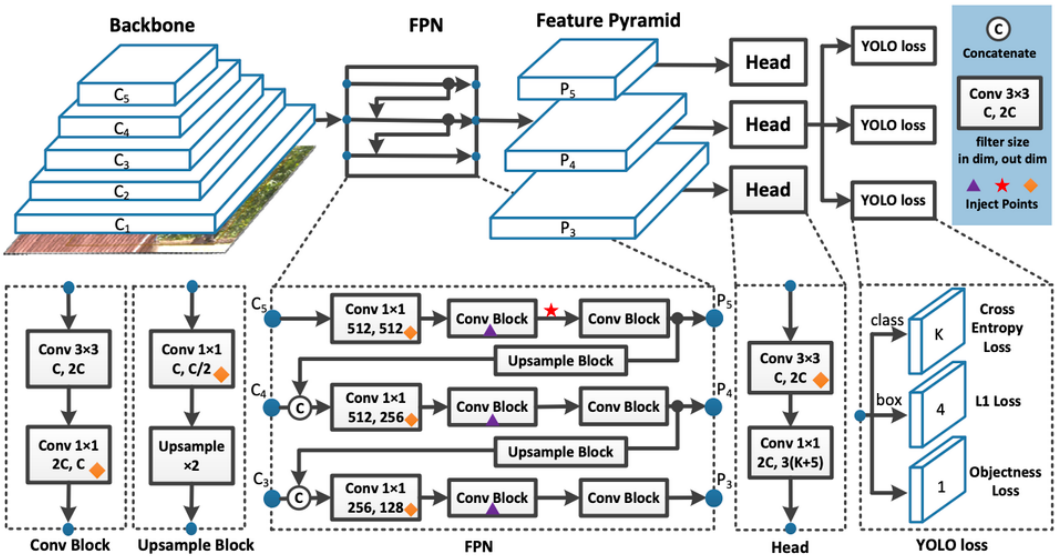


Figure 4-2 Architecture of YOLOv7

Figure 4-2 [37] shows the three primary modules of YOLOv7 architecture: Backbone, Neck, and Head.

- **Backbone:** An extended variant of ELAN serves as the backbone of YOLOv7. It lets the network go deeper without the computational cost ballooning. Hierarchical features get pulled from the input image through this structure, and gradient flow along with feature reuse stay intact throughout.
- **Neck:** YOLOv7's neck brings together a Path Aggregation Network (PAN) and ELAN-W modules. PAN fuses features across scales, which lifts detection accuracy for objects both small and large. ELAN-W then sharpens the spatial and contextual cues while features are being aggregated.
- **Head:** Object classes, bounding box coordinates (x, y, h, w), and objectness scores all come out of the head. Classification and regression are split into separate branches

through a decoupled head design, and this split helps convergence. Predictions land at multiple scales so objects of different sizes get picked up.

Several upgrades over earlier YOLO versions ship with YOLOv7, and the model keeps its compact size and fast inference speed. Bounding box coordinates are predicted through anchor-based regression, where the final box $B = (x, y, w, h)$ is worked out relative to the anchor box as follows:

$$x = \sigma(t_x) + c_x \tag{4-3}$$

$$y = \sigma(t_y) + c_y \tag{4-4}$$

$$w = p_w \cdot e^{t_w} \tag{4-5}$$

$$h = p_h \cdot e^{t_h} \tag{4-6}$$

where (t_x, t_y, t_w, t_h) are the predicted offsets, (c_x, c_y) is the top-left corner of the grid cell, and (p_w, p_h) are the dimensions of the anchor box.

On top of that, YOLOv7 packs in a Coarse-to-Fine Lead Head and Model Scaling, both of which shore up gradient flow and keep training stable. For label assignment, it uses dynamic-k matching to get tighter anchor-label associations. Data augmentation, which includes mosaic augmentation (stitching four images into one), Hue, Saturation, Value (HSV) color shifts, random flips, crops, and rescaling, runs automatically during training. Together these bulk up dataset variety and push the model to generalize better. Built-in early stopping is also there; if the validation loss (a mix of box loss, objectness loss, and classification loss) plateaus for a set number of epochs, training cuts off on its own to avoid overfitting.

4.1.3 Recognition of Component Values

In order to generate a complete and simulation-ready netlist, it is essential to not only detect the components themselves but also to extract their associated parameter values, which are generally annotated near the components in the circuit diagram. Thus, it becomes essential to integrate Optical Character Recognition (OCR) with the object detection pipeline to extract and map these values correctly to the corresponding components.

This involves identifying regions within the circuit image that likely contain text (done by YOLOv7), and then recognizing said texts in order to parse the component values along with their units.

The hand-written text recognition is done using a CRNN-based OCR system, following the design philosophy of Deep Text Recognition Benchmark (DTRB) framework, however minor changes deemed suitable for this project have been made to get the text detection and recognition pipeline.

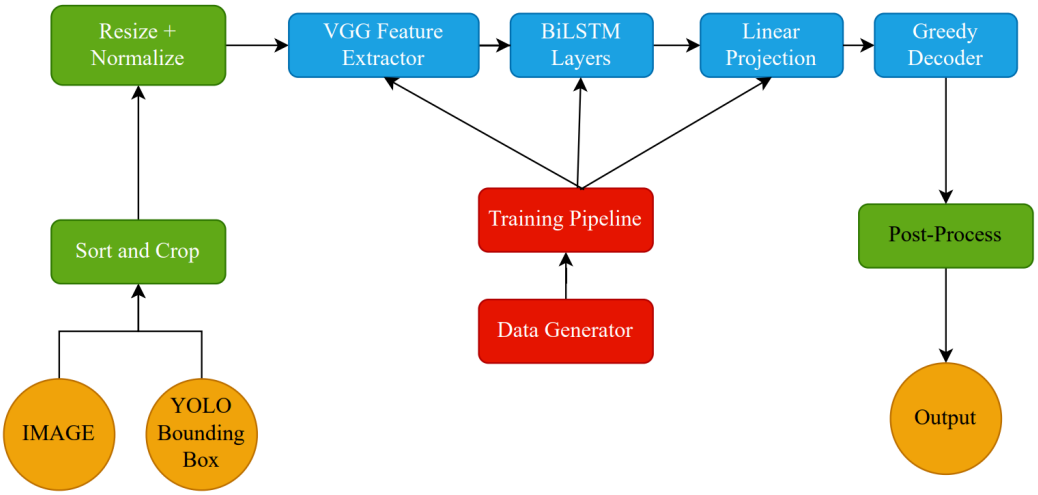


Figure 4-3 Text Detection and Recognition Pipeline

YOLOv7 outputs (bounding box information) help segregate textual information (class '0' in the used dataset) which are then cropped, resized (to image height as 64) and normalized for compatibility with text recognition module. This is then passed through to a CRNN architecture composed of a Convolutional Encoder, Sequence Modeling Engine with Linear Projection, and CTC Decoder as shown in Figure 4-3.

Convolutional Encoder (CNN): A CNN stack (e.g., VGG) transforms input image I from multi-dimensional $\mathbb{R}^{H\times W\times C}$ to feature maps:

$$F = f_{\text{CNN}}(I_i), \quad F \in \mathbb{R}^{H'\times W'\times C} \tag{4-7}$$

Sequence Modeling (Bidirectional LSTM): Each F becomes input to a bidirectional LSTM, yielding hidden states h_t. These hidden states capture context across the text line, allowing modeling of character sequences:

$$\overrightarrow{h_t} = \text{LSTM}_f(F_t, \overrightarrow{h_{t-1}}) \tag{4-8}$$

CTC Decoder: A softmax layer projects h_t over the character vocabulary as:

$$p_{t,k} = \frac{e^{W_k h_t + b_k}}{\sum_{k'} e^{W_{k'} h_t + b_{k'}}} \tag{4-9}$$

The probability of sequence z is computed over all alignments:

$$P(z \mid y) = \sum_{\pi : B(\pi) = z} \prod_{t=1}^{T} p_{t,\pi_t} \tag{4-10}$$

The CTC loss is given by the negative logarithm of $P(z \mid y)$. It enables end-to-end sequence decoding without explicit character bounding boxes.

During inference, greedy decoding strategy is used which selects the most-probable character at each time step while removing blank symbols and collapsing consecutive duplicate characters. The recognized text then needs to be mapped to the appropriate circuit component whose value it represents.

This project includes two separate OCR models for character recognition, the second one being TrOCR (TrOCR-small to be exact) developed by Microsoft, specifically for handwritten text.

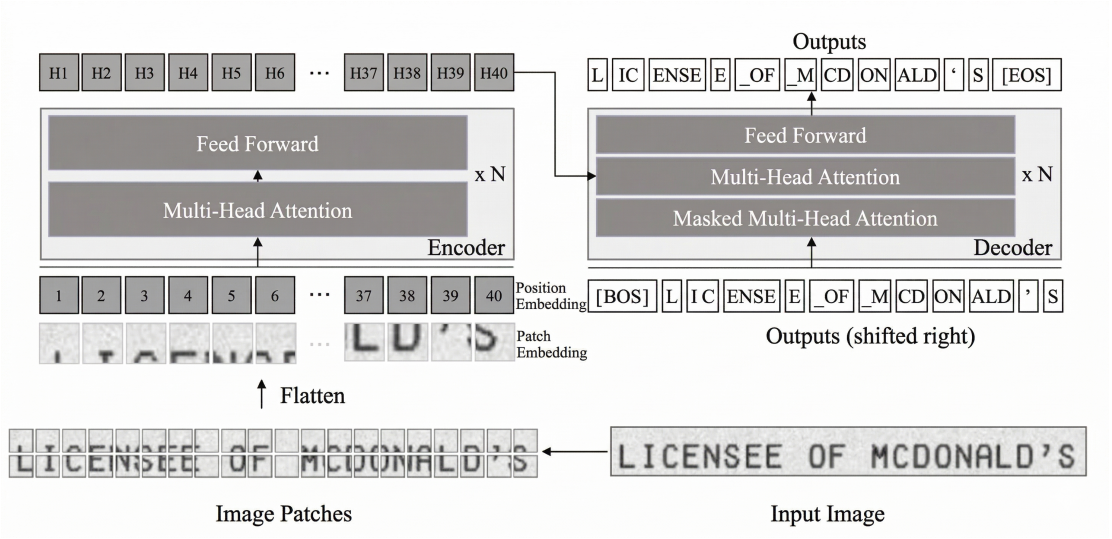


Figure 4-4 Architecture of TrOCR

Figure 4-4 [24] shows the encoder-decoder model (with pre-trained image transformer as encoder, and pre-trained text transformer as decoder) completing the basic architecture of TrOCR. The image transformer obtains the representaton of image patches, and likewise,

with the help of visual features and previous predictions, the text transformer generates the wordpiece sequence.

The process begins with the raw input image, denoted as $x_{img} \in \mathbb{R}^{3\times H_0\times W_0}$. The transformer architecture is designed to process sequences rather than raw pixel grids, therefore the image is first resized to a standardized dimension (H, W).

To format this data for the model, the image is decomposed into a series of N fixed-size square patches. For P as the patch side length, the total number of patches is given as:

$$N = \frac{HW}{P^2}$$

Every patch gets flattened into a vector and then linearly projected into a D-dimensional space; D here is the hidden dimension that stays constant across all transformer layers.

A `[CLS]` token is prepended to the sequence so that global information can be pooled into a single representation. If pre-trained DeiT models are used for initialization, a distillation token gets tacked on as well to help the model learn from a teacher. Transformers on their own have no built-in sense of where patches sit spatially (unlike CNNs), so learnable 1D position embeddings are summed into the patch embeddings to retain absolute positional cues. Viewing the image as a sequence of standalone patches lets the model spread its attention across fine-grained details or the broader layout as needed.

The decoder follows the standard transformer decoder blueprint for producing the final text sequence. It mirrors the encoder's stacked-layer design, though an encoder-decoder attention block is wedged in between the self-attention and feed-forward portions. Queries in this block come from the decoder's own internal state, and keys along with values are drawn from the encoder output. That arrangement lets the decoder zero in on whichever visual features matter most at each generation step.

A self-attention mask keeps the model from peeking at future tokens while training. The output at any position i therefore depends solely on the tokens before it. Hidden states from the decoder pass through a linear layer sized to the vocabulary V, and the resulting probability distribution over the vocabulary comes from the softmax function:

$$\sigma(h_{ij}) = \frac{e^{h_{ij}}}{\sum_{k=1}^{V} e^{h_{ik}}}$$

At inference time, beam search is used to explore the probability space and land on the most probable character sequence.

4.1.4 Wire Detection

Junction points picked up by the object detection model act as structural anchors for figuring out how the circuit is wired. These junctions first get converted from normalized detection coordinates into pixel space for geometric consistency. Every possible pair of junctions is then checked for potential wire connections.

A displacement vector and orientation angle are computed for each pair. An angular tolerance filter keeps only the near-horizontal and near-vertical pairs, tossing out diagonal ones to stay in line with typical schematic drawing conventions. The surviving pairs are snapped to strict axis-aligned representations, forming candidate wire segments.

To guard against detection noise, wire endpoints get quantized to a uniform grid and collinear segments are merged based on proximity and overlap. What comes out is a tidy, topologically consistent collection of horizontal and vertical wire segments that capture the underlying wiring.

4.1.5 Terminal Point Detection

The goal here is to locate the electrical connection points where wires meet components. Junction detections from the object detection model serve as the starting candidates. These cover wire endpoints and intersections (between wires as well as between wires and components).

With the final wire set in hand, terminal points are narrowed down by looking at where axis-aligned wire segments cross into component bounding boxes. Each such crossing marks a valid terminal where a wire touches a component edge, and the intersection conditions are solved analytically so that entry and exit points land precisely.

The resulting terminals tie wires and components together into a complete connectivity map, which feeds into circuit graph construction and the frontend display. All components of the project (CircuitJS-compatible text file generation, schematic editing, etc.) build on this representation downstream.

4.1.6 Integration Algorithm

From the individual outputs from each of the aforementioned modules, the task is now to generate a structured JSON representation of the circuit which has is to be used to generate a corresponding CircuitJS-compatible text file and also contribute in the schematic generation further down the line.

The integration between all these different outputs, in a simplistic form, would consist of the following steps:

- Get a list of components and their corresponding bounding box coordinates (from YOLOv7).
- Get a list of text data and their corresponding bounding box coordinates (from OCR).
- Map the component to the corresponding component value (text) based on the minimum Euclidean distance between a particular component and texts on the circuit.
- Once the terminals are determined based on the bounding boxes, match those terminal points to the nearest nodes in the circuit.
- Store the accumulated information in a JSON file.

4.1.7 CircuitJS-compatible Text File Generation

A circuitJS-compatible text filedescribes the connections between components in an electronic circuit. It acts as a blueprint, specifying which components are connected and how, without detailing the physical layout or specific component placement.

It is a text-based representation of an electrical circuit used for simulation by the CircuitJS engine. It provides all the necessary information about the components, their connections, and their electrical properties, enabling analysis such as transient response, DC/AC sweep, or frequency domain behavior.

The text file serves as input to a circuit simulation engine, which then performs numerical computations to evaluate how the circuit behaves under specified conditions.

For each component, this text file contains the information about its name, the nodes it is connected to on either side and the component's parameters or model (if necessary).

After the JSON file with abundant information about the components, their values, and the nodes they are connected to is generated, the next step is to convert said JSON file into a CircuitJS-compatible file (a `.txt` file which follows the appropriate conventions).

4.1.8 Digital Schematic Generation

The JSON file built by integrating all the information about the hand-drawn circuit is to be manipulated through different JavaScript libraries to generate a digital, editable schematic of the circuit. As briefly discussed in the Requirements section, react-draggable is used to map each component in JSON to a visual block (a symbol for the component as given by the KiCad symbol library). For each component, a graphical symbol needs to be rendered at a certain position and connection lines need to be drawn between the terminals and node positions. A rough pseudocode for the same is as follows:

Algorithm 1 Frontend Circuit Rendering

Require: Circuit JSON description J
Ensure: Rendered circuit schematic
  for all component $c \in J$.components **do**
    Draw schematic symbol of type c.type at position c.position
    Attach text label c.value near component position
  end for
  for all connection $k \in J$.connections **do**
    Retrieve terminal position from k.from and terminal index
    Retrieve node position from k.node
    Draw wire between terminal and node positions
  end for
  return Rendered schematic

4.1.9 Backend Development

The backend has been constructed using Next.js and FastAPI, functioning as the core middleware responsible for routing, and processing the system's ML pipeline and user interactions. Every endpoint follows a REST API design so that communication between frontend and server stays stateless.

When a hand-drawn circuit image arrives at an API endpoint like `/upload`, it first gets saved to temporary storage. From there, the server kicks off the YOLOv7 model (through a Python subprocess or a containerized inference API) to spot and localize electrical components. The OCR model runs in parallel to pull out textual annotations nearby. What comes back is a list of detected components with their bounding boxes alongside recognized texts carrying confidence scores and coordinates.

The backend also handles user interactions with the schematic. Deleting components, moving them around, adding wires, changing values; all of these go through dedicated API endpoints. Each edit gets written to a temporary database that keeps the circuit representation editable in real time.

4.1.10 LLM Integration

A large language model (LLM), DeepSeek v3.1 specifically, is integrated into the system to push circuit interaction beyond just GUI manipulation. This is what brings the 'analysis' side of the project to life. Users can ask things like "Explain the purpose of this resistor." or "Identify possible errors in this circuit." and get meaningful responses. The agent pulls contextual data from the backend (the circuit's JSON structure or CircuitJS file, for instance), assembles the prompt, and sends it off to the LLM over a secure API.

4.2 Line Detection

Two separate approaches to line (wire) detection were built and tested during this project. One relies on junction-point geometry paired with horizontal and vertical classification. The other traces paths on a skeletonized image using Multi-Head BFS. The results chapter puts both side by side; the BFS-based path-finding approach ended up being adopted for the final frontend system because it proved more accurate and held up better across varied inputs.

4.2.1 Line Detection via Horizontal and Vertical Classification

Wire detection under this approach starts from junction points that YOLOv7 has already identified. YOLO's detection coordinates sit in the unit square $[0, 1]^2$, so they first need to be denormalized into pixel space through the affine map $\mathcal{N}^{-1}(x_n, y_n) = (W \cdot x_n,\ H \cdot y_n)$. From there, every detected junction point is paired up with every other one. For a given pair $(\mathbf{j}_i, \mathbf{j}_k)$, the displacement vector $\mathbf{d} = \mathbf{j}_k - \mathbf{j}_i$ gets computed and the angular orientation $\tilde{\theta}$ is extracted using the four-quadrant arctangent.

Junction pairs are classified using an 18° tolerance zone:

- **Horizontal:** $\tilde{\theta} \in [0°, 18°] \cup [162°, 198°] \cup [342°, 360°]$
- **Vertical:** $\tilde{\theta} \in [72°, 108°] \cup [252°, 288°]$
- **Diagonal (rejected):** all remaining angles (60% of angular space)

Pairs that survive the filter are projected onto perfectly horizontal or vertical lines by averaging the relevant coordinate, then snapped to a 16-pixel grid. A merge algorithm consolidates overlapping collinear segments when they share the same orientation and fall within the grid tolerance.

To figure out where wires physically connect to components, the algorithm computes intersections between each detected wire segment and the component bounding boxes. Intersection conditions get checked at the pertinent box edges (left and right for horizontal wires, top and bottom for vertical ones), and each wire-box pair can yield up to two intersection points.

Pairwise complexity sits at $O(n^2)$, which gets unwieldy as junction count grows. Because only axis-aligned pairs pass the filter, a full 60% of angular space is thrown away, so angled or curved hand-drawn wires simply cannot be captured. Grid quantization alone introduces up to 11.3 pixels of positional drift. In dense regions, the merging heuristic sometimes fuses segments that belong to completely different wires. The deepest flaw, though, is that connections are guessed purely from where junctions happen to sit, with zero awareness of the actual wire paths in the image. Unrelated junctions that happen to

be axis-aligned end up linked by phantom wires. Terminal detection inherits all of these problems since it depends on the wire segments being correct in the first place.

4.2.2 Line Detection via Path Finding

The geometric approach's shortcomings called for something fundamentally different, so a skeleton-based path-finding method was put together. Instead of guessing connections from junction positions, this pipeline works straight on skeletonized circuit images, following the actual wire paths. At its core sits a Multi-Head Breadth-First Search (BFS) algorithm. The whole process splits into two stages, endpoint detection and graph construction.

Endpoint Detection: How endpoints are found depends on what class the object belongs to. Junction points (class 1) get a single endpoint: the bounding box center (x_c, y_c) is snapped to whichever skeleton pixel sits closest within an expanded search region (margin = 8 pixels). Components (class 2+) need two endpoints instead. Skeleton pixels lying on or near the bounding box border are scanned, with preference given to true topological endpoints (degree = 1). The pair with the greatest separation wins.

Graph Construction via Multi-Head BFS: A graph gets built with detected endpoints as nodes and skeleton connectivity supplying the edges. Every node claims a disk of radius r, and a `node_id_map` is precomputed so that each pixel knows which node owns it (or carries -1 if none does). BFS launches from each source node: the queue starts with skeleton pixels sitting at the disk boundary and crawls along the skeleton through 8-connectivity. The moment a pixel owned by some other node j turns up, the edge (i, j) is logged and that particular branch stops. Once the full graph is in place, self-edges get pruned. These are edges connecting two endpoints of the same bounding box, and stripping them out guarantees that only inter-object connections survive.

Advantages over horizontal/vertical classification: Because path-finding follows the actual wire paths visible in the image, phantom connections vanish. Wire orientation ceases to matter altogether; horizontal and vertical wires get handled just the same as diagonal ones. Wire detection and terminal detection collapse into one graph-construction step. Scaling is linear with skeleton pixel count instead of quadratic with junction count. For all of these reasons, the final deployed system runs on this approach.

4.2.3 Crossover Dissolution Algorithm

Hand-drawn circuit diagrams contain two visually similar yet electrically opposite situations i.e, a *crossover*, where two wires pass over each other without any electrical contact, and a *junction*, where wires genuinely meet. Getting the distinction wrong ruins

circuit reconstruction entirely. YOLOv7 flags crossovers as their own class. Multi-Head BFS (Section 4.2.2) then slots each one in as a single-point graph node, usually connected to four neighboring endpoints that correspond to the two wire segments crossing at that spot. The dissolution algorithm's job is to untangle these nodes into the correct topology.

Candidate Identification: A crossover node must have exactly four neighbors to be eligible for dissolution. Anything with fewer neighbors stays in the graph as a regular junction. This sidesteps problems where the detector mistakenly labels a junction as a crossover, or where a wire happens to dead-end right at the crossing.

Direction Vector Computation: Given a crossover node sitting at position $\mathbf{p}$, a unit direction vector pointing toward each of its four neighbors $\mathbf{n}_k$ is worked out as:

$$\hat{\mathbf{d}}_k = \frac{\mathbf{n}_k - \mathbf{p}}{\|\mathbf{n}_k - \mathbf{p}\|} \tag{4-11}$$

Optimal Pairing via Dot Product: Four neighbors can be split into two pairs in exactly three ways. Each candidate pairing $\{(\mathbf{n}_a, \mathbf{n}_b),\ (\mathbf{n}_c, \mathbf{n}_d)\}$ receives a score equal to the sum of dot products within its pairs:

$$S = \hat{\mathbf{d}}_a \cdot \hat{\mathbf{d}}_b + \hat{\mathbf{d}}_c \cdot \hat{\mathbf{d}}_d \tag{4-12}$$

Whichever pairing yields the most negative S wins. A strongly negative score means the paired neighbors sit on roughly opposite sides of the crossover, which is exactly the geometry expected when two wires cross.

Geometric Validation: Before anything gets rewired, both dot products in the chosen pairing have to clear a threshold: $\hat{\mathbf{d}}_a \cdot \hat{\mathbf{d}}_b < -0.7$. Passing this check confirms that each paired neighbor lies within roughly 45° of being directly across from the other through the crossover center.

Graph Transformation: When validation succeeds, the crossover node gets ripped out of the adjacency graph. Two fresh edges take its place, each linking one pair of opposite neighbors directly. What was a four-way junction now becomes two independent through-connections, faithfully modeling two wires that happen to cross paths without touching. Should the validation fail, the node simply remains as a junction so that electrical connectivity is preserved.

Once dissolution wraps up, the resulting graph feeds into final circuit assembly. Nodes in this graph stand for component terminals and junctions alongside wire endpoints; edges trace the actual wire paths. Crossover points no longer appear anywhere in the electrical topology.

4.3 Use-Case Diagram

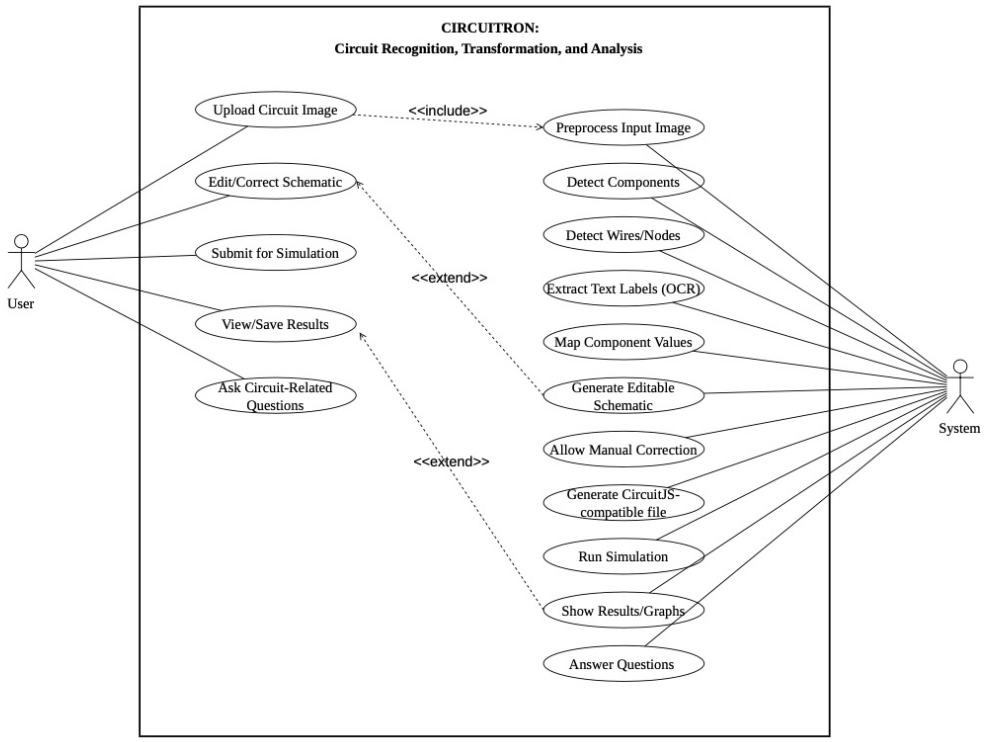


Figure 4-5 Use-Case Diagram for CIRCUITRON System

Figure 4-5 lays out how users interact with the CIRCUITRON system. The primary actor here is the user, whether that happens to be a student, an instructor, or a practicing engineer. Everything kicks off when the user uploads a hand-drawn circuit image.

As soon as the image lands, CIRCUITRON preprocesses it to sharpen clarity. YOLOv7 then picks out the components (e.g., resistors, capacitors) while junction-based wire detection maps out the wires and nodes. OCR runs at the same time, pulling text labels (e.g., component values) off the image and pairing them with the right components.

From the parsed circuit structure, an editable schematic gets generated. Users can step in at this point and manually fix any mistakes. Once they are satisfied, the schematic is converted into a CircuitJS-compatible text file and fed into CircuitJS for simulation. Voltage and current graphs come back as the output. An AI assistant is included as well for answering circuit-related questions. This may help in simply clearing up circuit-related questions and concepts, or also help in debugging.

In the diagram, the “includes” relationship flags sub-tasks that have to run every time, like component detection and OCR. “Extends” covers the optional bits, things like manual corrections or AI Q&A that add value but are not strictly necessary for the core workflow. Taken together, the diagram captures the full scope of what the system can do.

4.4 Performance Metrics Expectation

Even with careful training and fine-tuning, edge cases will slip through. Illegible handwriting or components that look nearly identical can trip up any model. That makes it essential to pin down performance metrics ahead of time so there is a clear threshold for when a model qualifies as "good-enough." Table 4-1 lists the metrics chosen for this project along with the target values; once those targets are hit, training can be halted.

Table 4-1 Expected Performance Metric Values for Individual Modules

Pipeline Stage	**Performance Metric**	**Target Value/Expectation**
Component Detection	mAP@0.5	0.95
Text Recognition	Character Accuracy	0.65

4.4.1 Component Detection Metric

mAP@0.5 (mean Average Precision at an IoU threshold of 0.5) gauges how accurately the model spots components while getting both the class label and the bounding box right. Average Precision (AP) is computed per class first, and then averaged across every class. Setting the IoU threshold at 0.5 means a predicted box counts as correct only when its overlap with the ground truth box reaches at least 50%.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{4-13}$$

When mAP@0.5 is high, it signals that detections are landing accurately and that few components are being missed or mislabeled.

4.4.2 Text Recognition Metric

Character Accuracy tallies how many individual characters in the predicted text line up with the ground truth:

$$\text{Character Accuracy} = \frac{\text{Number of Correct Characters}}{\text{Total Number of Characters in Ground Truth}} \tag{4-14}$$

Component values on circuit diagrams ($10k$, 100Ω, $1\mu F$, and so on) leave zero room for misreads. Even one wrong character, say confusing 'k' with '1', throws off the entire simulation. OCR has to nail these consistently, because reliable value-to-component mapping hinges on high character accuracy.

4.5 System Flowchart

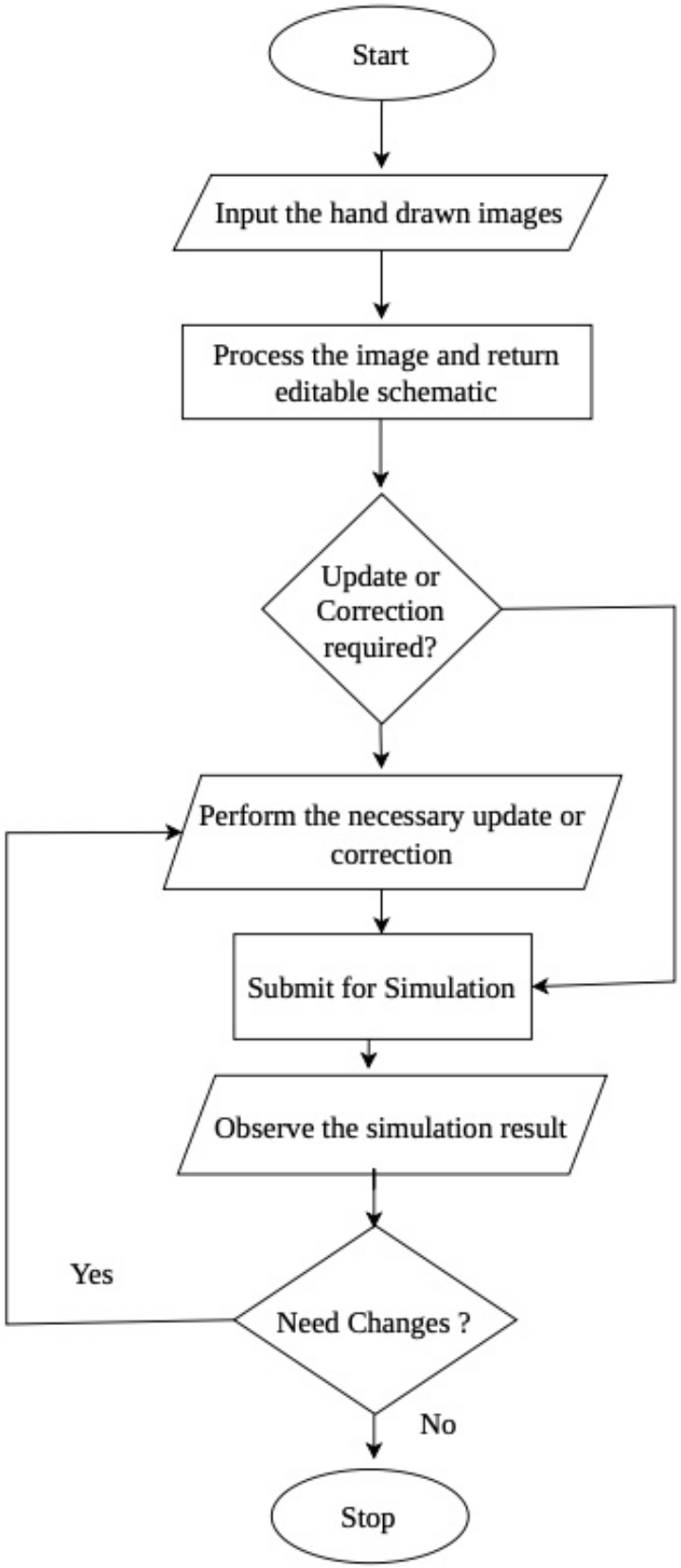


Figure 4-6 System Flowchart

The key architectural modules involved in the working of the system have been demonstrated by the block diagram in Figure 4-1. Now, the flowchart in Figure 4-6 demonstrates the motions a user needs to go through to use the final product. Once the image of a hand-drawn circuit is uploaded, the system processes the image (internally using the techniques explained in the previous sections) and returns an editable schematic. Any necessary update or correction may be performed if necessary and then the schematic can be submitted for simulation and analysis. Relevant graphs are returned after analysis.

4.6 Dataset Analysis

In order to develop the system using the system for automated circuit diagram understanding, the Circuit Graph Handwritten Diagrams (CGHD) (Circuit Graph Handwritten Diagrams) dataset is used. This dataset provides a comprehensive suite of ground truth annotations tailored to facilitate key computer vision tasks such as object detection, instance segmentation, and text recognition. While additional datasets (such as circuitVQA, Digitize-HCD datasets) may be used to fine-tune the models in the future to achieve even higher values for performance metrics, as of yet, only the CGHD dataset has been used.

4.6.1 Dataset Overview

The CGHD dataset comprises 4.99 GB of annotated data spanning over 3,000 raw images. The data is separated into folders named drafters, going from -1 through 31, and each drafter folder contains hand-sketches of 12 unique circuit designs, with each circuit drawn twice and captured under four different conditions (varying angles, lighting, and minor distortions), resulting in up to 96 images per drafter. As is convention, the dataset is to be split into (70-10-20)% of the entire data for training, validation and testing respectively.

Table 4-2 Summary of Data in the CGHD Dataset

Items	**Number**
Distinct Circuits	372
Raw Images	3,269
Bounding Box Annotations	248,020
Text String Annotations	85,417 (93.74% completeness)
Character Types	98
Object Classes	61

Every image is rigorously annotated and the annotations provided per image include:

- **Bounding Boxes (PASCAL VOC format)** for 61 electrical component classes, including resistors, capacitors, transistors, operational amplifiers, and textual elements.
- **Text Labels** that indicate component values and identifiers (e.g., "$10k\Omega$", "$C1$", "V").
- **Instance Segmentation Polygons** (LabelMe JSON format) for pixel-level object outlines.
- **Binary Segmentation Maps** that differentiate circuit strokes (foreground) from background artifacts.
- **Partial Netlist Files** (in `.asc` format) for some samples; useful for validating circuit reconstruction.

The text annotation coverage is extensive, with labels ranging from single characters (like "R" or "$R1$") to full descriptors (like "Resistor", "Variable Resistor", etc.).

4.6.2 Class-wise Data Statistics

Table 4-3 Frequency of Each Class with its Code in the CGHD Dataset

Code	Class Name	Count	Code	Class Name	Count
1	text	91,122	32	opamp.schmitt	16
2	junction	79,202	33	optocoupler	162
3	crossover	10,180	34	ic	1,506
4	terminal	11,253	35	ic.ne555	262
5	gnd	5,501	36	ic.v_regulator	362
6	vss	1,354	37	xor	162
7	voltage.dc	335	38	and	860
8	voltage.ac	292	39	or	440
9	voltage.battery	765	40	not	496
10	resistor	14,400	41	nand	228
11	resistor.adj	1,052	42	nor	59
12	resistor.photo	65	43	probe	184
13	cap.unpol	5,841	44	probe.curr	72
14	cap.pol	2,227	45	probe.volt	32
15	cap.adj	88	46	switch	2,184
16	inductor	1,976	47	relay	89
17	inductor.ferrite	120	48	socket	184
18	inductor.coupled	10	49	fuse	226
19	transformer	483	50	speaker	410
20	diode	3,890	51	motor	108
21	diode.led	1,212	52	lamp	301
22	diode.thyrector	16	53	microphone	72
23	diode.zener	609	54	antenna	24
24	diac	40	55	crystal	112
25	triac	98	56	mechanical	32
26	thyristor	415	57	magnetic	364
27	varistor	184	58	optical	127
28	transistor.bjt	3,678	59	block	0
29	transistor.fet	977	60	explanatory	610
30	transistor.photo	177	61	unknown	32
31	opamp	752			

5 IMPLEMENTATION DETAILS

The implementation of the system to convert hand-drawn circuit diagrams into simulation-ready netlists and digital schematics follows a modular approach, aligning with the pipeline described in the architecture and methodology.

5.1 Dataset Manipulation and Preprocessing

5.1.1 Conversion of Annotation Formats

To accommodate the multiple models in the pipeline (YOLOv7 for component detection and OCR for value extraction), annotation files provided in the CGHD dataset in Pascal VOC (XML) format were converted into compatible formats. Likewise, for EasyOCR-inspired custom-trained OCR, all text-sections needed to be cropped out from the image with corresponding labels (ground truth) and then fed to the fine-tuned model.

In Pascal VOC format, the information regarding the circuit and its components are stored in the following format as shown by the example below:

```
<annotation>
    <folder>images</folder>
    <filename>C1_D1_P2.jpg</filename>
    <path>./drafter_1/images/C1_D1_P2.jpg</path>
    <source>
        <database>CGHD</database>
    </source>
    <size>
        <width>3024</width>
        <height>4032</height>
        <depth>3</depth>
    </size>
    <segmented>0</segmented>
    <object>
        <name>voltage.dc</name>
        <pose>Unspecified</pose>
        <truncated>0</truncated>
        <difficult>0</difficult>
        <bndbox>
            <xmin>327</xmin>
            <ymin>546</ymin>
            <xmax>604</xmax>
            <ymax>838</ymax>
```

```
            <rotation>10</rotation>
        </bndbox>
    </object>
</annotation>
```

For YOLO-based detection, XML annotations were converted to YOLO format (`.txt`) using a custom Python script. Each `.txt` file contains the class index and normalized bounding box coordinates in the format:

```
<class_id> <x_center> <y_center> <width> <height>
```

The pseudocode for the VOC to YOLO annotation conversion is as follows:

Algorithm 2 VOC to YOLO Annotation Conversion

Require: XML annotation file X, class map C, image width W, image height H
Ensure: List of YOLO formatted annotation lines Y

```
Parse XML file X into tree structure
Y ← ∅
for all object nodes o in XML root do
    clsname ← class name of o
    if clsname ∉ C then
        continue
    end if
    clsid ← C[clsname]
    Extract bounding box (x_min, y_min, x_max, y_max) from o
    Convert VOC coordinates to normalized YOLO format
    x_center ← (x_min + x_max) / (2W)
    y_center ← (y_min + y_max) / (2H)
    w ← (x_max − x_min) / W
    h ← (y_max − y_min) / H
    Append formatted string (x_center, y_center, w, h) to Y
end for
return Y
```

The calculations run for this conversion is as follows:

$$x_{\text{center}} = \frac{327 + 604}{2 \times 3024} = \frac{465.5}{3024} = 0.1539 \tag{5-1}$$

$$y_{\text{center}} = \frac{546 + 838}{2 \times 4032} = \frac{692}{4032} = 0.1716 \tag{5-2}$$

$$w = \frac{604 - 327}{3024} = \frac{277}{3024} = 0.0916 \tag{5-3}$$

$$h = \frac{838 - 546}{4032} = \frac{292}{4032} = 0.0724 \tag{5-4}$$

This gives the information required by YOLO in the appropriate format. It is convention to save this new annotation file in the same directory as the image.

Likewise, for OCR compatibility, the same Pascal VOC annotations were converted to Lightning Memory-Mapped Database (LMDB) format. LMDB is a high-speed key-value store where each key corresponds to an index and each value contains an image-text pair (serialized).

The pseudocode for the conversion of Pascal VOC format to OCR-compatible LMDB file format is as follows:

Algorithm 3 Image Cropping and LMDB Dataset Creation

Require: Input image file I, crop coordinates $(y_{\min}, y_{\max}, x_{\min}, x_{\max})$, target size (W, H)
Ensure: LMDB database containing image-label pairs
  Load image I into memory
  Crop image using coordinates $[y_{\min} : y_{\max}, x_{\min} : x_{\max}]$
  Resize cropped image to size (W, H)
  Convert cropped image to byte representation
  Convert image array to PIL image
  Initialize in-memory byte buffer
  Save image to buffer in JPEG format
  Extract byte stream from buffer
  Store image and label in LMDB
  Open LMDB environment with predefined map size
  Begin write transaction
  Store image bytes with key `image-00001`
  Store corresponding label with key `label-00001`
  Store total sample count with key `num-samples`
  Commit transaction
  return LMDB environment

After this script is run in its entirety, a file in LMDB format, as is expected by OCR, is received as shown in the example below:

```
Key: b'image-00001'
Value: b'\xff\xd8\xff\xe0\x00\x10JFIF...'

Key: b'label-00001'
Value: b'voltage.dc'

Key: b'num-samples'
Value: b'000001'
```

After necessary conversion, the dataset was ready for being plugged into the required models for fine-tuning them. But first, some experimentation was done with different forms of image augmentation techniques.

5.1.2 Image Augmentation

Since the dataset contains handwritten circuit diagrams captured under variable lighting and orientations, several augmentation techniques were applied to improve model generalization including Contrast Enhancement (global) and Adaptive Brightness Adjustment by factors of 0.8 and 1.2. Furthermore, geometric augmentation techniques, like spatial flipping (both horizontally and vertically) and rotation (e.g., 90° or random rotation) proved to be unwise operations for this project. For example, if a "C" is rotated by 90°, the "C" now resembles a "U", and the same applies to flipping ("M" might look like "W", for example). Rather than increasing accuracy, it was realized that using rotation or flipping would simply confuse the model.

5.1.3 Fine-tuning YOLOv5 and YOLOv7 for Component Detection

YOLOv5 was initially used for benchmarking, and later upgraded to YOLOv7 for better accuracy and inference speed (comparison shown in the Results section). Training and fine-tuning were done using the CGHD dataset after converting into YOLO-compatible format.

The training parameters employed are as follows:

- **Number of classes:** 61
- **Training split:** 70% training, 10% validation, 20% testing
- **Epochs:** 200 (for YOLOv5), 100 (for YOLOv7)
- **Image size:** 416×416 pixels
- **Batch size:** 16 (for YOLOv5), 32 (for YOLOv7)
- **Optimizer:** Stochastic Gradient Descent (SGD) (Stochastic Gradient Descent)
- **Learning rate:** 0.01 (initial)
- **Augmentations used in training:** Mosaic Augmentation, HSV Augmentation, Random rotating/flipping

5.1.4 Class Consolidation and Retraining

Satisfactory results were not obtained through the initial 61-class approach. To improve the performance, consolidation of 61 classes into 15 was done, and the models were retrained on this new taxonomy.

Table 5-1 Consolidation of Original Classes into Unified YOLO Classes

Class Code	Class Name	Consolidated Classes
0	capacitor	capacitor.adjustable, capacitor.polarized, capacitor.unpolarized
1	crossover	crossover
2	diode	diode, diode.light_emitting, diode.thyrector, diode.zener
3	gnd	gnd
4	inductor	inductor, inductor.coupled, inductor.ferrite
5	integrated_circuit	integrated_circuit, integrated_circuit.ne555, integrated_circuit.voltage_regulator
6	junction	junction
7	operational_amplifier	operational_amplifier, operational_amplifier.schmitt_trigger
8	resistor	resistor, resistor.adjustable, resistor.photo
9	switch	switch
10	terminal	terminal
11	text	text
12	transistor	transistor.bjt, transistor.fet, transistor.photo
13	voltage	voltage.ac, voltage.battery, voltage.dc
14	vss	vss

A merge mapping was applied to the original annotations, grouping visually similar subclasses (e.g., capacitor.polarized and capacitor.unpolarized into capacitor) as shown in the Table 5-1. This consolidation is done not only considering visual similarity but also based on the usage i.e, the resulting 15 classes make up the majority of basic-level circuits Four YOLO architectures were then trained on this 15-class dataset for benchmarking:

- **YOLOv7 (retrained):** 100 epochs, batch size 16, image size 640×640
- **YOLOv8:** 100 epochs, batch size 16, image size 640×640
- **YOLOv11:** 100 epochs, batch size 16, image size 640×640
- **YOLOv26:** 100 epochs, batch size 16, image size 640×640

All models were trained on identical data splits with the same augmentation pipeline, confidence threshold of 0.25, and IoU threshold of 0.45.

5.1.5 Implementation of Custom Text Recognition (OCR)

To extract textual information such as resistor values, capacitor labels, and other component annotations from the circuit diagrams, a CRNN-based custom OCR model was trained, and implemented. The input image in cropped form (cropped for each text component) was passed through this model after training, which returned the content of

each detected text element. The position of said text element was found using bounding boxes from YOLOv7. This allowed us to localize and read numerical values or labels directly from the circuit.

Three different modules, each for feature extraction, sequence modeling, and prediction were trained based on the DTRB framework. Initially, simply EasyOCR was also attempted; however, this didn't yield usable results, ultimately requiring training from scratch. The training, which was done for 27 epochs, had early stopping as it's accuracy per epoch failed to achieve even a 0.01% increase within 5 epochs. Other training details include:

- **Training epochs:** 27 (with early stopping)
- **Image Height:** 64px (fixed for all croppings)
- **Aspect Ratio:** 4:1 (allowing for variable width)
- **Learning Rate:** 0.01 (initial)

5.1.6 Fine-tuning TrOCR for Text Recognition

Following the initial custom CRNN approach, a second OCR model was implemented using Microsoft's TrOCR-small-printed architecture. Unlike the CRNN model which was trained from scratch, TrOCR was fine-tuned from pre-trained weights, leveraging transfer learning from large-scale printed text datasets.

The fine-tuning was performed on cleaned text crops extracted from the CGHD dataset. All text crops from a single image were processed in one forward pass, and per-token log-probability was averaged across the generated sequence with following details:

- **Base model:** Microsoft TrOCR-small-printed (pre-trained)
- **Fine-tuning epochs:** 3
- **Checkpoint:** Epoch 2 (best validation performance)
- **Inference precision:** float16 (half-precision for GPU acceleration)
- **Maximum output tokens:** 16 (sufficient for values like "10kΩ", "100μF")

The TrOCR model is integrated into the pipeline through a singleton service that lazy-loads the processor and model weights when it is first invoked.

5.1.7 Line Detection Algorithm Implementation

Circuit diagrams carry an inherent geometric regularity: wires tend to run either horizontally or vertically, and each wire bridges two distinct components. The line detection algorithm leans on that regularity. Where the Probabilistic Hough Transform

(Probabilistic Hough Transform (PHT)) works solely off pixel gradients, this method instead takes the junction points that YOLOv7 has already picked up and treats them as anchor nodes for rebuilding the wiring layout through geometry.

Every unique pair of junctions coming out of the object detection model gets examined. For each pair, the angle of the connecting vector is calculated. Hand-drawn circuits are rarely pristine, so an angular tolerance of $\pm 18^\circ$ was baked in to sort connections into horizontal or vertical buckets. Segment coordinates were then "snapped" onto a 16-pixel grid to keep the whole diagram geometrically consistent.

Algorithm 4 Wire Classification and Alignment

Require: Set of junctions J, angular tolerance $\omega_{\text{tol}} = 18^\circ$, grid size $g = 16$
Ensure: Set of raw wire segments S
  $S \leftarrow \emptyset$
  for all pairs $(j_a, j_b) \in \text{Combinations}(J, 2)$ **do**
    $\Delta x \leftarrow j_b.x - j_a.x$
    $\Delta y \leftarrow j_b.y - j_a.y$
    $\theta \leftarrow \text{atan2}(\Delta y, \Delta x)$
    if $|\theta| \le \omega_{\text{tol}} \vee |\theta - 180^\circ| \le \omega_{\text{tol}}$ **then**
      Horizontal alignment check
      $y_{\text{avg}} \leftarrow \frac{j_a.y + j_b.y}{2}$
      $y_{\text{snap}} \leftarrow \text{Round}\left(\frac{y_{\text{avg}}}{g}\right) \times g$
      $S \leftarrow S \cup \{(H, y_{\text{snap}}, \min(j_a.x, j_b.x), \max(j_a.x, j_b.x))\}$
    else if $|\theta - 90^\circ| \le \omega_{\text{tol}} \vee |\theta - 270^\circ| \le \omega_{\text{tol}}$ **then**
      Vertical alignment check
      $x_{\text{avg}} \leftarrow \frac{j_a.x + j_b.x}{2}$
      $x_{\text{snap}} \leftarrow \text{Round}\left(\frac{x_{\text{avg}}}{g}\right) \times g$
      $S \leftarrow S \cup \{(V, x_{\text{snap}}, \min(j_a.y, j_b.y), \max(j_a.y, j_b.y))\}$
    end if
  end for
  return S

Pairwise comparison has a predictable side effect: redundant overlapping segments pile up for the same wire. Three junctions sitting on one line, for example, yield three separate segments (A-B, B-C, and A-C) when really only one continuous wire exists. A collinear segment merging step was added to clean this up.

Segments are first split by orientation (horizontal versus vertical) and sorted along their primary axis. Overlapping or neighboring segments then get folded together. During each merge, the axis coordinate of whichever segment is longer takes priority, since a longer stroke is a more trustworthy indicator of where the wire actually runs.

The wire–component intersection detection algorithm pinpoints where detected wire segments meet component bounding boxes. YOLOv7 annotations supply the component

Algorithm 5 Collinear Segment Merging

Require: List of segments L, gap tolerance ω
Ensure: List of merged segments M

```
Sort L by primary axis, then by starting coordinate
M ← ∅
curr ← L[0]
for all next ∈ L[1 . . . N] do
    if |curr.axis − next.axis| ≤ ω ∧ next.start ≤ curr.end + ω then
        curr.start ← min(curr.start, next.start)
        curr.end ← max(curr.end, next.end)
        if Length(next) > Length(curr) then
            Preserve axis of the longer segment
            curr.axis ← next.axis
        end if
    else
        M ← M ∪ {curr}
        curr ← next
    end if
end for
M ← M ∪ {curr}
return M
```

regions, which then get translated into pixel coordinates. Every horizontal or vertical wire is checked against each component box's edges for geometric intersection. Those intersection points pin down how the circuit is actually wired together, and the frontend can optionally render them for visual verification.

5.2 Line Detection via Path Finding

The algorithm behind the path-finding line detection pipeline first splits the work into a handful of stages. First, a spatial map of node regions is assembled. Multi-head BFS traversal then crawls through skeleton pixels to uncover which nodes connect to which. Spurious self-edges get pruned in a final cleanup pass.

Building a node identification map kicks off the whole process. The map gives constant-time answers to a simple question: does this pixel sit inside a node region? A circular disk of radius r is stamped around every detected endpoint, and each pixel under that disk is tagged with the owning node's index. Algorithm 7 lays out the formal construction.

Algorithm 6 Wire–Component Intersection Detection

Require: Wire segments W, YOLO label file L, image width I_w, image height I_h
Ensure: Set of intersection points P

```
Parse label file L
Convert normalized bounding boxes to pixel coordinates
Discard non-component classes
Store component boxes B
Detect intersections between wires and components
P ← ∅
for all wire w ∈ W do
    for all box b ∈ B do
        if w is horizontal and crosses left or right edge of b then
            Add intersection point to P
        else if w is vertical and crosses top or bottom edge of b then
            Add intersection point to P
        end if
    end for
end for
Draw wires, component boxes, and intersection points on image
Save output visualization
return P
```

Algorithm 7 Build Node ID Map

Require: Skeleton image S of size $H \times W$, list of nodes $\mathcal{N} = \{(x_0, y_0), \ldots, (x_{n-1}, y_{n-1})\}$, radius r
Ensure: Node ID map M of size $H \times W$

```
M ← array of size H × W initialized to −1
D ← DiskMask(r)
for i ← 0 to n − 1 do
    (n_x, n_y) ← N[i]
    for each (dy, dx) ∈ D do
        (p_y, p_x) ← (n_y + dy, n_x + dx)
        if 0 ≤ p_y < H and 0 ≤ p_x < W then
            M[p_y][p_x] ← i
        end if
    end for
end for
return M
```

Once the node ID map is constructed, the algorithm performs a breadth-first search from each node to discover its neighbors along the skeleton. The BFS starts from the boundary of each node's disk region and traverses skeleton pixels using 8-connectivity. When this BFS traversal meets a pixel belonging to a different node, that node is recorded as a neighbor which terminates the branch. This ensures that each skeleton path is followed until it reaches another endpoint. Algorithm 8 describes this procedure.

Algorithm 8 BFS Neighbors for Node

Require: Node index i, nodes $\mathcal{N}$, skeleton S, node ID map M, radius r
Ensure: Set of neighbor node indices $\mathcal{A}_i$

$(n_x, n_y) \leftarrow \mathcal{N}[i]$
$\mathcal{A}_i \leftarrow \emptyset$
$V \leftarrow$ boolean array of size $H \times W$ initialized to `false`
$\mathcal{D} \leftarrow \text{DiskMask}(r)$
for each $(dy, dx) \in \mathcal{D}$ **do**
    $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$
    if $0 \le p_y < H$ **and** $0 \le p_x < W$ **then**
        $V[p_y][p_x] \leftarrow$ `true`
    end if
end for
$Q \leftarrow$ empty queue
for each $(dy, dx) \in \mathcal{D}$ **do**
    $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$
    for each $(n'_y, n'_x) \in \text{Neighbors8}(p_y, p_x)$ **do**
        if $\neg V[n'_y][n'_x]$ **and** $S[n'_y][n'_x] > 0$ **then**
            $V[n'_y][n'_x] \leftarrow$ `true`
            Q.enqueue$((n'_y, n'_x))$
        end if
    end for
end for
while Q is not empty **do**
    $(c_y, c_x) \leftarrow Q$.dequeue()
    $j \leftarrow M[c_y][c_x]$
    if $j \neq -1$ **and** $j \neq i$ **then**
        $\mathcal{A}_i \leftarrow \mathcal{A}_i \cup \{j\}$
        continue
    end if
    for each $(n'_y, n'_x) \in \text{Neighbors8}(c_y, c_x)$ **do**
        if $\neg V[n'_y][n'_x]$ **and** $S[n'_y][n'_x] > 0$ **then**
            $V[n'_y][n'_x] \leftarrow$ `true`
            Q.enqueue$((n'_y, n'_x))$
        end if
    end for
end while
return $\mathcal{A}_i$

The final step removes spurious connections between endpoints that belong to the same detected object. Since line segment bounding boxes produce two endpoints each, the BFS may incorrectly connect these as neighbors via the skeleton segment within the bounding box. By maintaining a list of endpoint pairs from the same object, these self-edges are explicitly removed from the adjacency graph. Algorithm 9 implements this filtering.

Algorithm 9 Remove Self-Edges (Same-Object Connections)

Require: Adjacency $\mathcal{G}$, endpoint pairs $\mathcal{P} = \{(p_1^{(k)}, p_2^{(k)})\}_{k=1}^{m}$, nodes $\mathcal{N}$
Ensure: Filtered adjacency $\mathcal{G}'$

$\mathcal{G}' \leftarrow \mathcal{G}$
for each $(p_1, p_2) \in \mathcal{P}$ **do**
    $i \leftarrow \text{FindIndex}(p_1, \mathcal{N})$
    $j \leftarrow \text{FindIndex}(p_2, \mathcal{N})$
    if $i \neq -1$ **and** $j \neq -1$ **then**
        $\mathcal{G}'[i] \leftarrow \mathcal{G}'[i] \setminus \{j\}$
        $\mathcal{G}'[j] \leftarrow \mathcal{G}'[j] \setminus \{i\}$
    end if
end for
return $\mathcal{G}'$

5.2.1 Complexity of Implemented Algorithm

This section analyzes the computational complexity of the line detection pipeline. Let H, W denote the image dimensions, n the number of nodes (endpoints), r the disk radius for node regions, and P the number of skeleton pixels (typically $P \ll H \times W$).

Table 5-2 Time complexity breakdown for Multi-Head BFS algorithm

Operation	**Complexity**	**Description**
Node ID Map Construction	$O(n \cdot r^2)$	Each node marks πr^2 pixels
Single Node BFS	$O(P)$	Each skeleton pixel visited at most once
Complete Adjacency Build	$O(n \cdot P)$	BFS from each of n nodes
Self-Edge Removal	$O(m)$	m = number of endpoint pairs
Total	$O(n \cdot r^2 + n \cdot P)$	Dominated by BFS traversals

Table 5-2 summarizes the time complexity of each operation in the pipeline.

Table 5-3 Space complexity breakdown for Multi-Head BFS algorithm

Data Structure	**Space**	**Description**
Skeleton Image S	$O(H \times W)$	Binary image storage
Node ID Map M	$O(H \times W)$	Integer array for node mapping
Visited Array V	$O(H \times W)$	Per-BFS boolean array
BFS Queue Q	$O(P)$	Maximum skeleton pixels in queue
Adjacency $\mathcal{G}$	$O(n + E)$	E = number of edges
Disk Offsets $\mathcal{D}$	$O(r^2)$	Precomputed disk mask
Total	$O(H \times W + n + E)$	Dominated by image arrays

Table 5-3 details the memory requirements of the algorithm.

The visited array can be reused across BFS calls with a reset, or alternatively, a hash set can be used for sparse skeletons to reduce visited storage to $O(P)$. For typical skeleton images where $P = O(H + W)$ (linear in image perimeter), the overall complexity is:

$$T(n, H, W, r) = O(n \cdot r^2 + n \cdot (H + W)) = O(n \cdot (r^2 + H + W))$$

In the real world, this approach can be considered efficient because it scales with the number of nodes, but it is bound by the physical resolution of the image sensor.

5.2.2 Crossover Dissolution

After the adjacency graph is constructed by Multi-Head BFS, crossover nodes must be resolved into their true topology. A crossover represents two wires crossing without connecting, and is initially represented as a single graph node with four neighbors. After identifying these nodes, the dissolution algorithm replaces every node with two direct connections. This removes the crossover from the electrical topology.

Algorithm 10 Crossover Dissolution Algorithm

Require: Adjacency graph $\mathcal{G}$, node positions $\mathbf{p}$, detection results $\mathcal{R}$
Ensure: Modified graph $\mathcal{G}$ with crossovers dissolved

for each detection $r \in \mathcal{R}$ with $r.\text{cls} = 1$ **do**
  $a \leftarrow$ node index of r's endpoint in $\mathbf{p}$
  if $a = -1$ **or** $|\mathcal{G}[a]| \neq 4$ **then**
    continue
  end if
  $\{n_1, n_2, n_3, n_4\} \leftarrow \mathcal{G}[a]$
  for each neighbor n_k **do**
    $\hat{\mathbf{d}}_k \leftarrow \dfrac{\mathbf{p}[n_k] - \mathbf{p}[a]}{\|\mathbf{p}[n_k] - \mathbf{p}[a]\|}$
  end for
  $S^* \leftarrow +\infty, \quad \Pi^* \leftarrow \varnothing$
  for each pairing $\Pi = \{(n_i, n_j), (n_k, n_l)\}$ of the 4 neighbors into 2 pairs **do**
    $S \leftarrow \hat{\mathbf{d}}_i \cdot \hat{\mathbf{d}}_j + \hat{\mathbf{d}}_k \cdot \hat{\mathbf{d}}_l$
    if $S < S^*$ **then**
      $S^* \leftarrow S, \quad \Pi^* \leftarrow \Pi$
    end if
  end for
  $\{(n_a, n_b), (n_c, n_d)\} \leftarrow \Pi^*$
  if $\hat{\mathbf{d}}_a \cdot \hat{\mathbf{d}}_b > -0.7$ **or** $\hat{\mathbf{d}}_c \cdot \hat{\mathbf{d}}_d > -0.7$ **then**
    continue
  end if
  for each neighbor $n_k \in \{n_a, n_b, n_c, n_d\}$ **do**
    $\mathcal{G}[n_k] \leftarrow \mathcal{G}[n_k] \setminus \{a\}$
  end for
  Remove a from $\mathcal{G}$
  Add edges (n_a, n_b) and (n_c, n_d) to $\mathcal{G}$
end for
return $\mathcal{G}$

All three ways of pairing the four neighbors are tried. Whichever pairing yields the smallest summed dot product wins, because that is the arrangement where paired neighbors point most nearly in opposite directions. A dot-product cutoff of -0.7 (roughly 45° away from perfect opposition) acts as a sanity check on whether the geometry genuinely looks like two wires crossing. Any crossover node that falls short of this bar, whether it has fewer than four neighbors or its geometry is too skewed, simply stays in the graph as an ordinary junction.

5.3 Frontend and Backend Implementation

The frontend was built on top of React.js alongside CircuitJS, with auxiliary packages like react-draggable and the KiCad Symbol Library pulled in where needed. On the server side, the circuitron backend exposes a REST API that oversees the full lifecycle

of circuit simulations. Through this API, users can submit CircuitJS-compatible text files, have the circuit syntax checked, or kick off simulations under transient, AC, or DC analysis modes.

Every simulation request receives its own unique identifier. That identifier lets the frontend fire off a job and come back later to check on it, since dedicated status endpoints handle progress tracking asynchronously. Finished simulation data (time-domain voltages, currents, and so on) can be fetched through a separate results endpoint, while administrative tasks like halting a running simulation or wiping old records are covered by their own routes too. The API speaks in structured JSON throughout and sticks to standard Hypertext Transfer Protocol (HTTP) status codes, which keeps error reporting predictable and makes hooking the frontend up to it straightforward.

5.3.1 Frontend-Backend Interaction

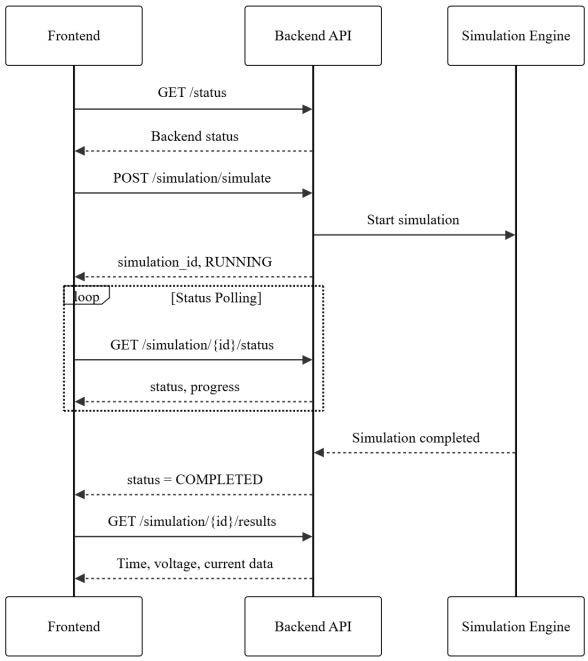


Figure 5-1 Interaction Diagram for the Web Application

All communication between the frontend and the backend occurs over HTTP, hitting the documented endpoints. When a user modifies the editable circuit schematic, those inputs get translated into CircuitJS-compatible text files and forwarded to the simulation endpoint. Progress is tracked by having the frontend poll the status endpoint at regular intervals. Once the simulation wraps up, the results are fetched and rendered visually through CircuitJS graphs alongside React-based interactive components.

5.4 Circuit Simulation using CircuitJS

Internally, CircuitJS translates the scanned circuit into a set of matrix-based mathematical equations rather than treating it as a drawing of wires. What it actually constructs is a numerical model of the circuit, essentially a network graph suited for computation. A time-domain simulation loop then solves these equations over successive iterations. Convergence for non-linear components relies on iterative numerical methods. Because CircuitJS is embedded directly in the frontend, users can design their circuits, run animated simulations, and inspect outputs all within the browser, with no extra software to install.

5.5 Error Handling

The overall error in this project is a summation of errors from the number of individual modules working together. These individual errors propagate from one module to another and accumulate. To handle errors in such a case, foremost the errors in the output of the individual modules need to be controlled and only then can the accumulated error be kept in check. The performance metrics tabulated in the previous section are to be strictly adhered to if there's any hope of generating the CircuitJS compatible text file (and digital schematic) with nominal errors.

For the reduction of false matches and OCR noise, post-processing techniques such as value format validation, confidence filtering, and text cleanup can be implemented. Value format validation ensures only strings that match known (and appropriate) electrical units, say $10k$, $1\mu F$, $5V$ are retained. Confidence filtering helps discard OCR results with low confidence scores, and text cleanup normalizes ambiguous symbols such as converting "O" to "0", "μ" to "u".

A basic pseudocode for sample value format validation is as follows:

Algorithm 11 Validation and Filtering of OCR Component Values

Require: OCR detected text entries T
Ensure: Filtered list of valid component values V
  Define regular expression patterns for resistance, capacitance, and voltage
  $V \leftarrow \emptyset$
  for all text entry $t \in T$ **do**
    if t.text matches any defined pattern **then**
      Append t to V
    end if
  end for
  return V

The list of components shown here is in no way meant to be exhaustive; rather it is to show how this pseudocode ensures that only valid electrical component values are used in the final component-to-value mapping and schematic generation.

To prevent propagating error (error from the output of YOLOv7 bleeding into OCR, for instance), a combined confidence threshold checking mechanism can be implemented which marks an item as uncertain if the confidence score from YOLOv7 is less than a certain value (e.g., 0.6) and the same from OCR is less than a certain value (e.g., 0.85). For more robustness, a human-in-the-loop mechanism can be implemented which prompts the user to manually approve or correct OCR-predicted values, fix connection errors, etc. This can be implemented to trigger when the confidence score is below the set threshold or when other error handling mechanisms such as value format validation fail. The pseudocode for this is as follows:

Algorithm 12 Confidence-Based Error Checking and Validation

Require: Module outputs O, confidence threshold ω, format validation function f
Ensure: Valid outputs V and uncertain outputs U

```
V ← ∅
U ← ∅
for all item o ∈ O do
    if o.confidence ≥ ω ∧ f(o.value) then
        Append o to V
    else
        Append o to U
    end if
end for
if U ≠ ∅ then
    Trigger manual override
end if
return (V, U)
```

6 RESULTS AND ANALYSIS

6.1 Comparative Analysis of Object Detection using YOLOv5 and YOLOv7

6.1.1 Experimental Setup

A comparative study using YOLOv5 and YOLOv7 to detect and classify hand-drawn electronic circuit components was conducted to determine which model is better-suited for the needs of this project. The dataset consisted of 61 classes, including resistors, capacitors (polarized and unpolarized), diodes, transistors (Bipolar Junction Transistor (BJT) and Field-Effect Transistor (FET)), integrated circuits, switches, and more. To ensure a fair comparison, both models were trained and validated on identical data splits.

6.1.2 Performance Metrics

mAP vs Number of Epochs

In order to monitor the progress in models' ability to recognize components as the training progressed, mAP@0.5 was plotted across all epochs. These curves depict how quickly and effectively each model learns over time.

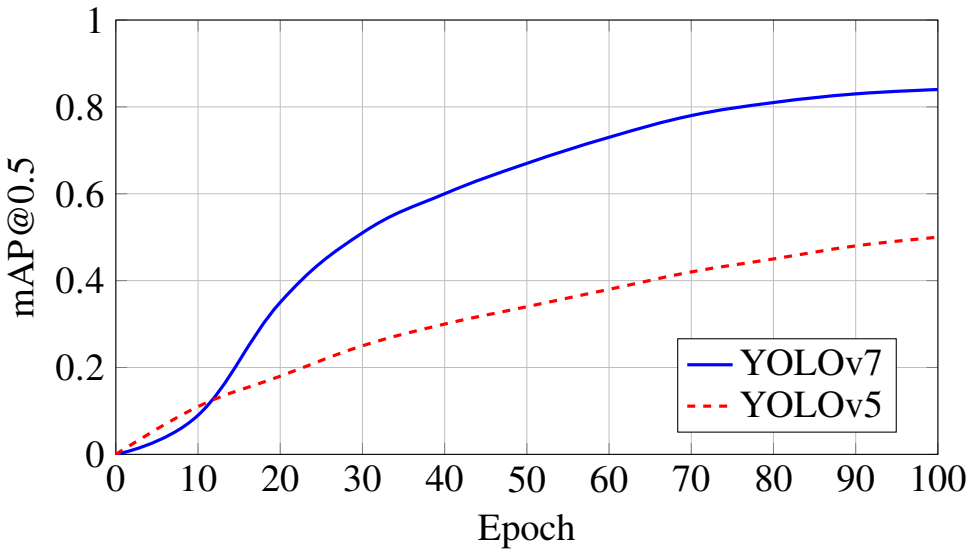
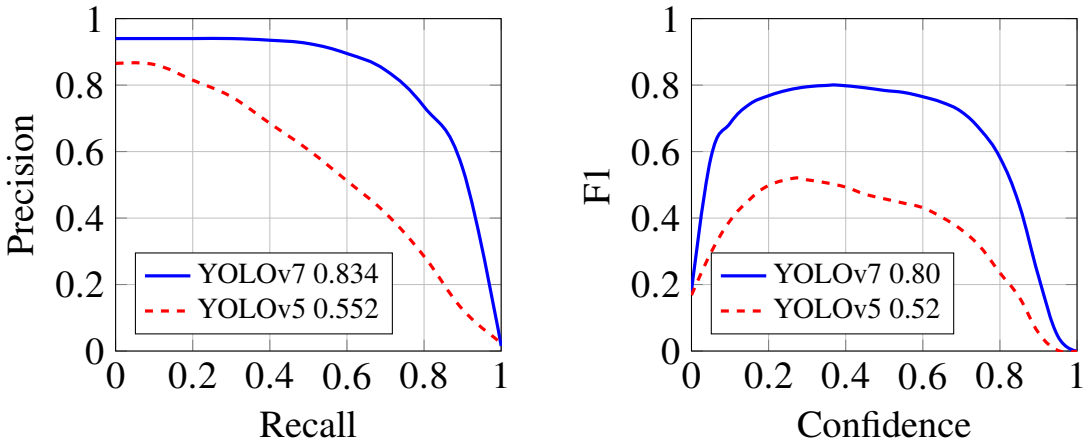


Figure 6-1 mAP@0.5 comparison over 100 training epochs.

It is clear from Figure 6-1 that the mAP score for YOLOv7 are significantly better than that for YOLOv5.

Precision vs Recall and F1 Score vs Confidence

Figure 6-2 demonstrate the F1 Score-Confidence Curve for YOLOv5 and YOLOv7 respectively. The larger the area under this curve, the higher the robustness of the precision-recall trade-off across thresholds.



(a) Precision-Recall curve.

(b) F1-Confidence curve.

Figure 6-2 Performance metrics comparison: (a) Precision-Recall and (b) F1-Confidence curves.

Figure 6-2 also demonstrate the Precision-Recall Curve for YOLOv5 and YOLOv7 respectively. The area under this curve gives the average precision (AP) which informs how good the model is at ranking positive samples above negative ones. The area under each of the curves in these figures is higher for YOLOv7 in comparison to YOLOv5. This is expected behavior and proves YOLOv7 is better than YOLOv5 when it comes to performance.

6.1.3 Quantitative Results Comparison

Final epoch results show YOLOv7 outperformed YOLOv5 across all major metrics.

Table 6-1 Comparison of Performance Metrics for YOLOv5 and YOLOv7

Metric	**YOLOv5**	**YOLOv7**
mAP@0.5	0.61	0.83
mAP@0.5:0.95	0.41	0.60
Precision	0.84	0.93
Recall	0.62	0.78
Epochs Trained	200	100

Overall, YOLOv7 showed a clear advantage over YOLOv5 in our experiments. It showed faster convergence and higher values for precision, recall, mAP@0.5, and mAP@0.5:0.95. It also required fewer epochs for convergence. Additionally, YOLOv7's confusion matrix also showed better class separation with fewer misclassifications. These improvements make YOLOv7 a more robust choice for this project's object detection needs.

6.2 Class Taxonomy Consolidation

The fine-grained component subclasses like capacitor.polarized and capacitor.unpolarized looked nearly indistinguishable to the detector, which bred persistent inter-class confusion in case of 61-class approach. Merging visually overlapping subclasses brought the taxonomy down to 15 classes. Four YOLO architectures (YOLOv7, YOLOv8, YOLOv11, YOLOv26) were then trained and benchmarked on this consolidated dataset, and their numbers are contrasted here against the earlier 61-class results.

For the sake of a fair comparison, every model was trained and evaluated on the 15-class dataset under identical conditions. The data splits stayed the same across all four architectures, with a confidence threshold fixed at 0.25 and an IoU threshold of 0.45 during evaluation.

6.2.1 Training and Validation Loss: 15-Class Models

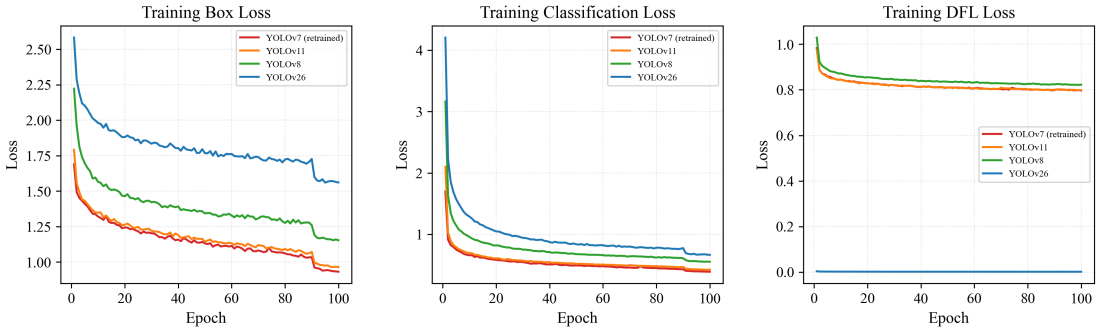


Figure 6-3 Training loss comparison across four YOLO architectures

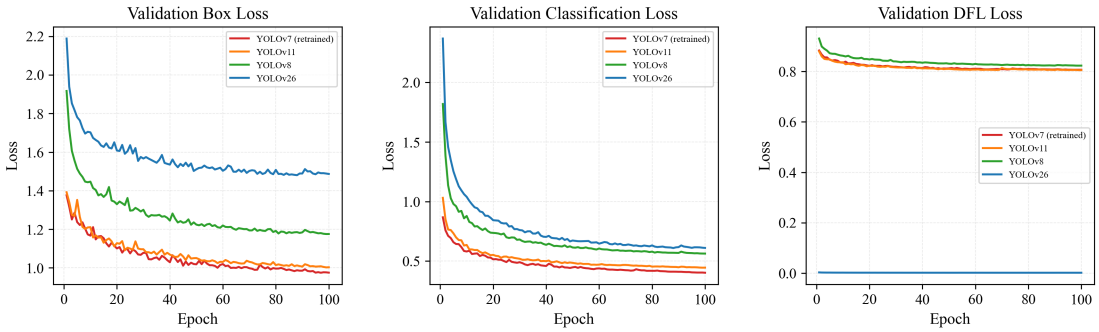


Figure 6-4 Validation loss comparison across four YOLO architectures

Figure 6-3 and Figure 6-4 lay out the training and validation losses (box, classification, DFL) for all four models. YOLOv7 (retrained) and YOLOv11 drove their training losses down noticeably faster than the other two, which points to stronger representational capacity. On the validation side, those same two architectures settled at the lowest

final loss values, suggesting they generalize more reliably and resist overfitting on this particular dataset.

6.2.2 Performance Metrics: 15-Class Models

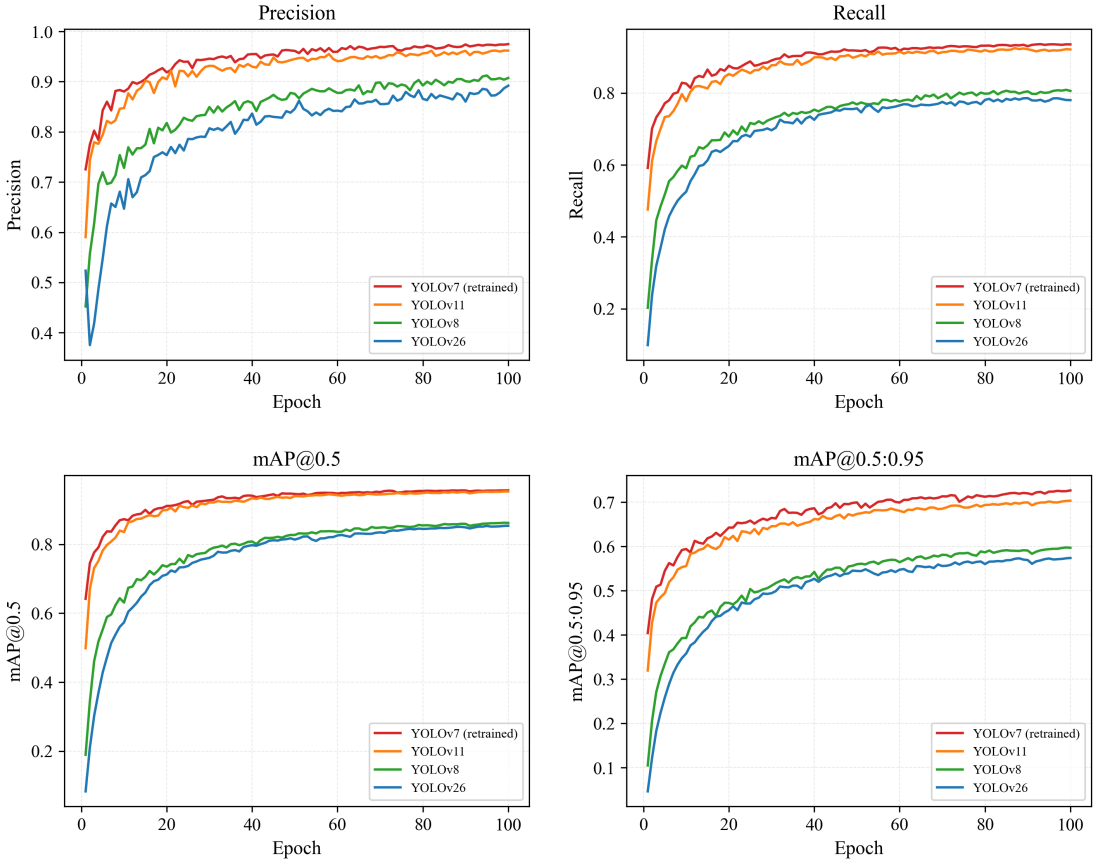


Figure 6-5 Performance metrics vs. epochs for four YOLO architectures on the 15-class dataset

Figure 6-5 plots all four performance metrics over 100 training epochs for each architecture. YOLOv7 (retrained) and YOLOv11 climb quickly, hitting near-peak values around epoch 40 to 50 and holding steady from there. YOLOv8 and YOLOv26 converge at a considerably slower pace, plateauing well below the other two, especially in recall and mAP@0.5:0.95.

6.2.3 Quantitative Results Comparison for 15-Class Dataset

Table 6-2 Performance comparison of four YOLO architectures on the 15-class dataset (final epoch)

Metric	YOLOv7 (retrained)	YOLOv11	YOLOv8	YOLOv26
Precision	**0.9747**	0.9617	0.9073	0.8919
Recall	**0.9350**	0.9209	0.8058	0.7801
mAP@0.5	**0.9562**	0.9531	0.8620	0.8537
mAP@0.5:0.95	**0.7266**	0.7033	0.5969	0.5738
Epochs	100	100	100	100

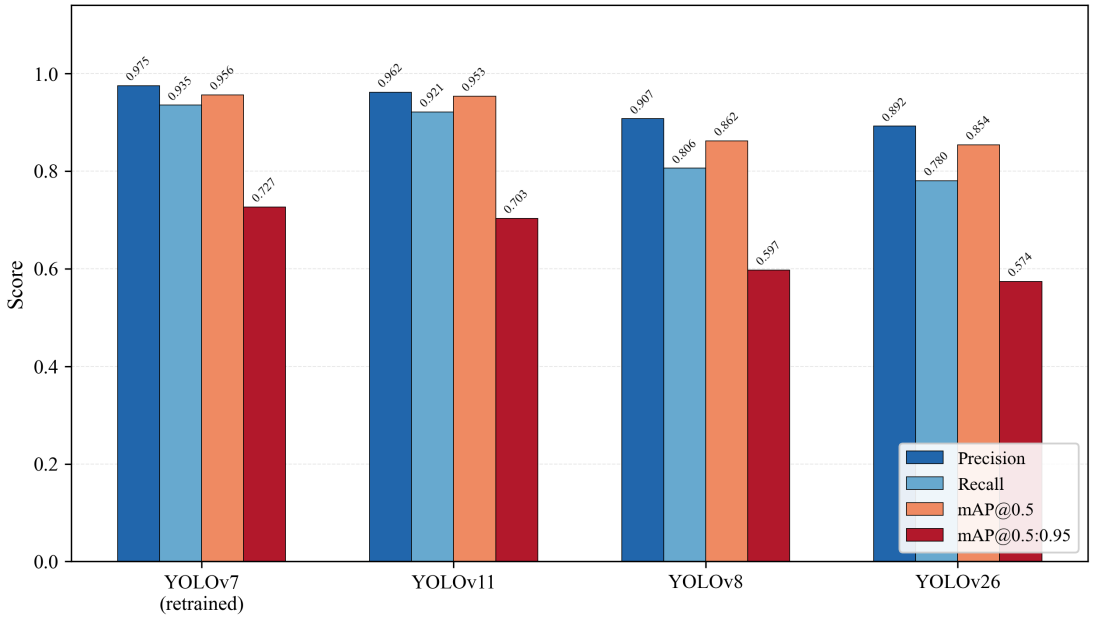


Figure 6-6 Final-epoch performance metrics for four YOLO architectures on the 15-class dataset

Table 6-2 collects the final-epoch numbers for all four architectures on the 15-class dataset; Figure 6-6 renders the same data as a grouped bar chart. YOLOv7 (retrained) tops every metric, with YOLOv11 trailing close behind. Both of these models clear the 95% mark in mAP@0.5, which speaks to their strong detection capability on the merged class set. YOLOv8 and YOLOv26 fall roughly 10 points short in mAP@0.5 and about 13 points short in mAP@0.5:0.95, which makes them a weaker fit for this project's requirements.

6.2.4 Impact of Class Taxonomy Consolidation

Table 6-3 Impact of class taxonomy consolidation on YOLOv7 performance

Metric	61-Class	15-Class	Improvement
Precision	0.93	0.9747	+4.8%
Recall	0.78	0.9350	+19.9%
mAP@0.5	0.83	0.9562	+15.2%
mAP@0.5:0.95	0.60	0.7266	+21.1%

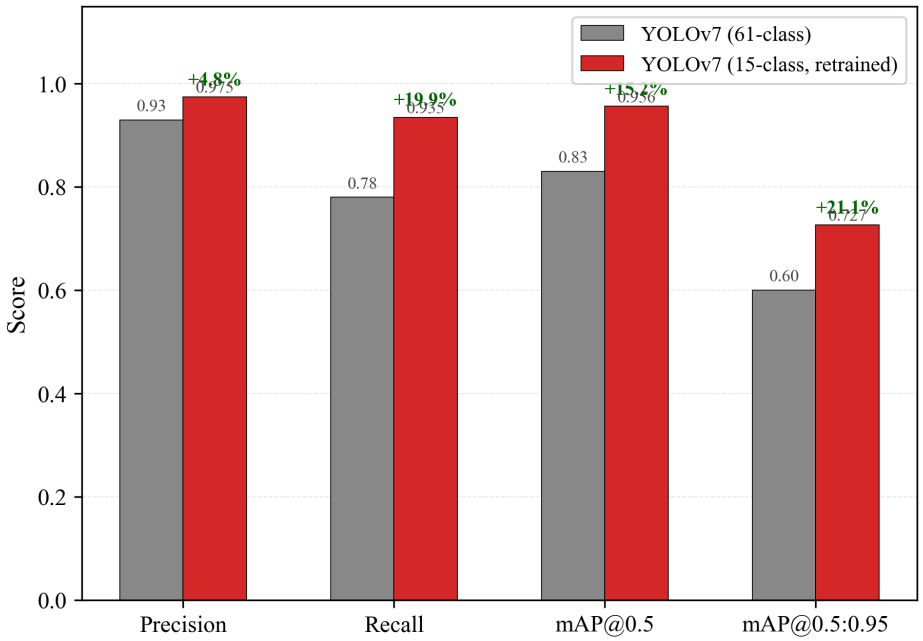


Figure 6-7 Effect of class taxonomy consolidation on YOLOv7: 61-class vs. 15-class performance

To put a number on what the taxonomy reduction actually bought, YOLOv7 was compared under both the 61-class and 15-class setups. Figure 6-7 and Table 6-3 capture these results. The gains were sizable: precision rose by 4.8%, recall jumped 19.9%, mAP@0.5 climbed 15.2%, and mAP@0.5:0.95 saw the steepest lift at 21.1%. Taken together, these figures confirm that the original fine-grained taxonomy was injecting avoidable inter-class confusion, dragging detection quality down.

6.3 Holistic Object Detection Comparison

Table 6-4 Unified comparison of all object detection configurations

Model	Classes	Precision	Recall	mAP@0.5	mAP@0.5:0.95
YOLOv5	61	0.84	0.62	0.61	0.41
YOLOv7	61	0.93	0.78	0.83	0.60
YOLOv8	15	0.9073	0.8058	0.8620	0.5969
YOLOv26	15	0.8919	0.7801	0.8537	0.5738
YOLOv11	15	0.9617	0.9209	0.9531	0.7033
YOLOv7 (retrained)	**15**	**0.9747**	**0.9350**	**0.9562**	**0.7266**

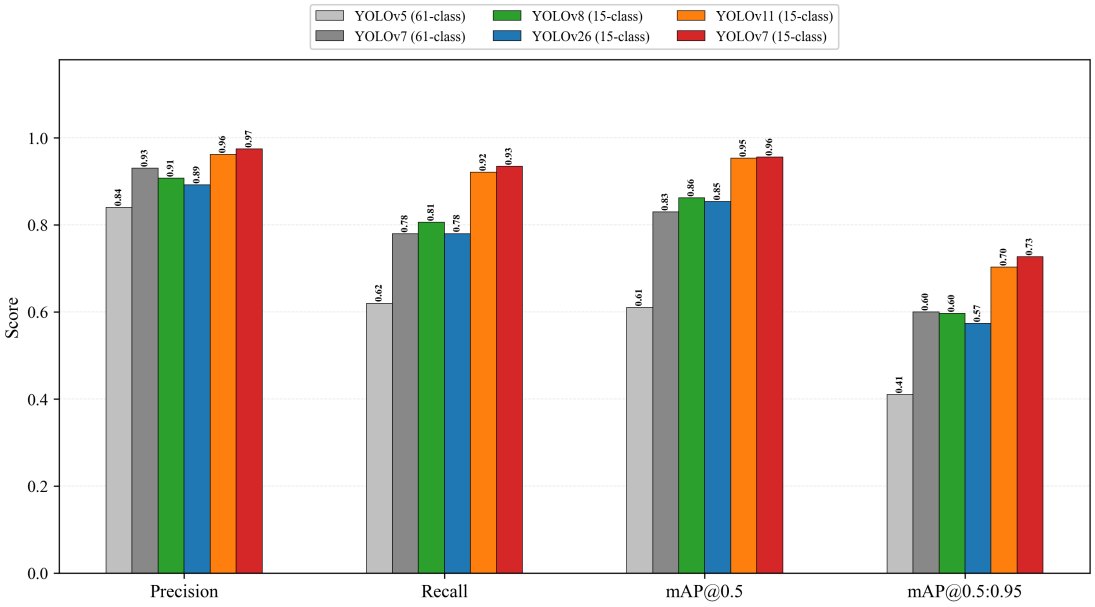


Figure 6-8 Grouped bar chart comparing all six detection configurations across Precision, Recall, mAP@0.5, and mAP@0.5:0.95

All six detection configurations evaluated over the course of this project are brought together in this section, covering two class taxonomies and five architectures. Table 6-4 and Figure 6-8 aggregate these results side by side.

6.3.1 Object Detection Performance Evolution

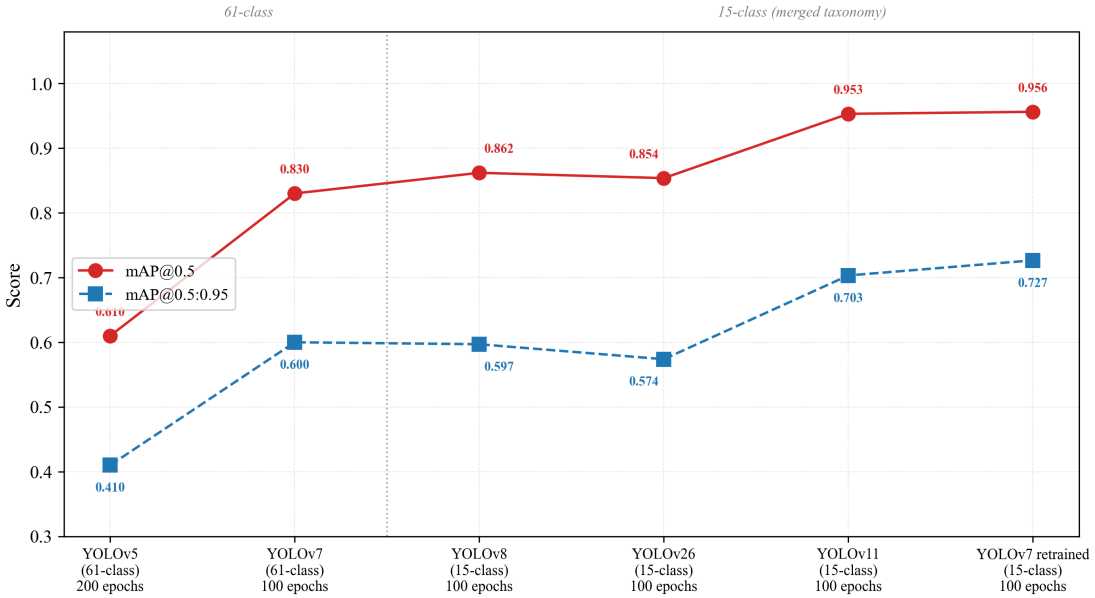


Figure 6-9 Performance evolution of object detection across all configurations showing mAP@0.5 and mAP@0.5:0.95

Figure 6-9 maps how detection performance shifted over the project timeline. Improvement came in two waves: first, upgrading from YOLOv5 to YOLOv7 while still on the 61-class taxonomy, and second, shrinking the taxonomy to 15 classes and pitting several architectures against one another. The retrained YOLOv7 on 15 classes sits at the top, with mAP@0.5 climbing from 0.61 (YOLOv5, 61-class) to 0.9562, amounting to a cumulative 56.8% gain.

6.3.2 Radar Visualization of Top Models

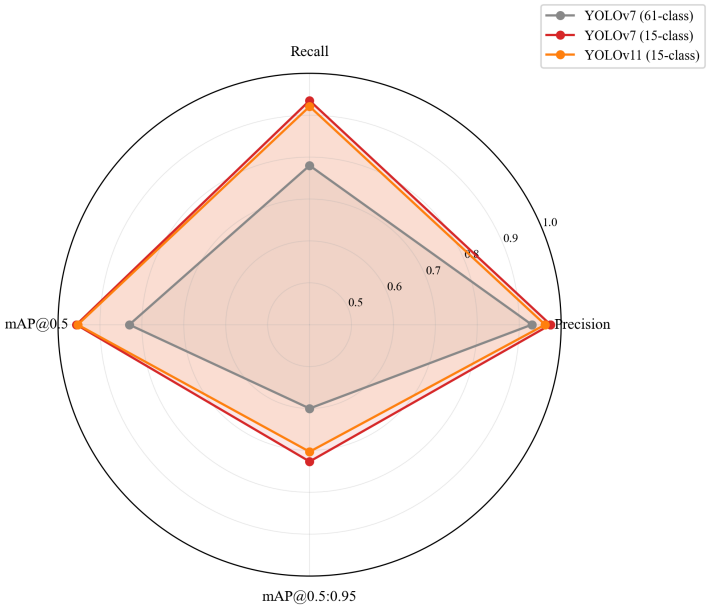


Figure 6-10 Radar chart comparing the three best detection configurations across four performance metrics

Figure 6-10 renders a radar chart of the three strongest configurations. YOLOv7 (61-class) lags visibly in recall (0.78 versus 0.93+), whereas both 15-class models fill out nearly the entire performance envelope. The retrained YOLOv7 (15-class) marginally outperforms YOLOv11 across all axes, making it the model of choice for the final pipeline.

Overall, from all these findings, it is evident that YOLOv7 (retrained on the 15-class taxonomy) is the optimal object detection model for this project. This also goes to show that class taxonomy might matter more than the architecture at times.

6.3.3 Confusion Matrix for the retrained YOLOv7

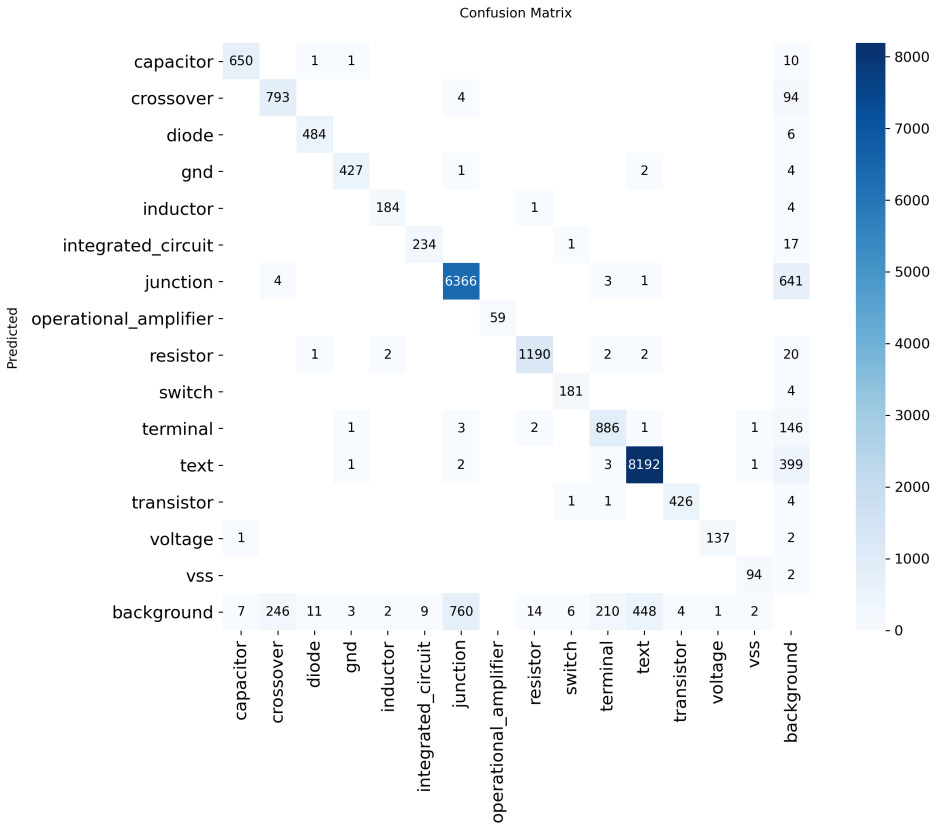


Figure 6-11 Confusion Matrix for the retrained YOLOv7

Figure 6-11 demonstrates the confusion matrix for the retrained YOLOv7 (15 classes). It has a darker main diagonal and very light off-diagonals, which means that the model exhibits strong class separability with minimal inter-class confusion.

6.4 Retrained YOLOv7 Demonstration

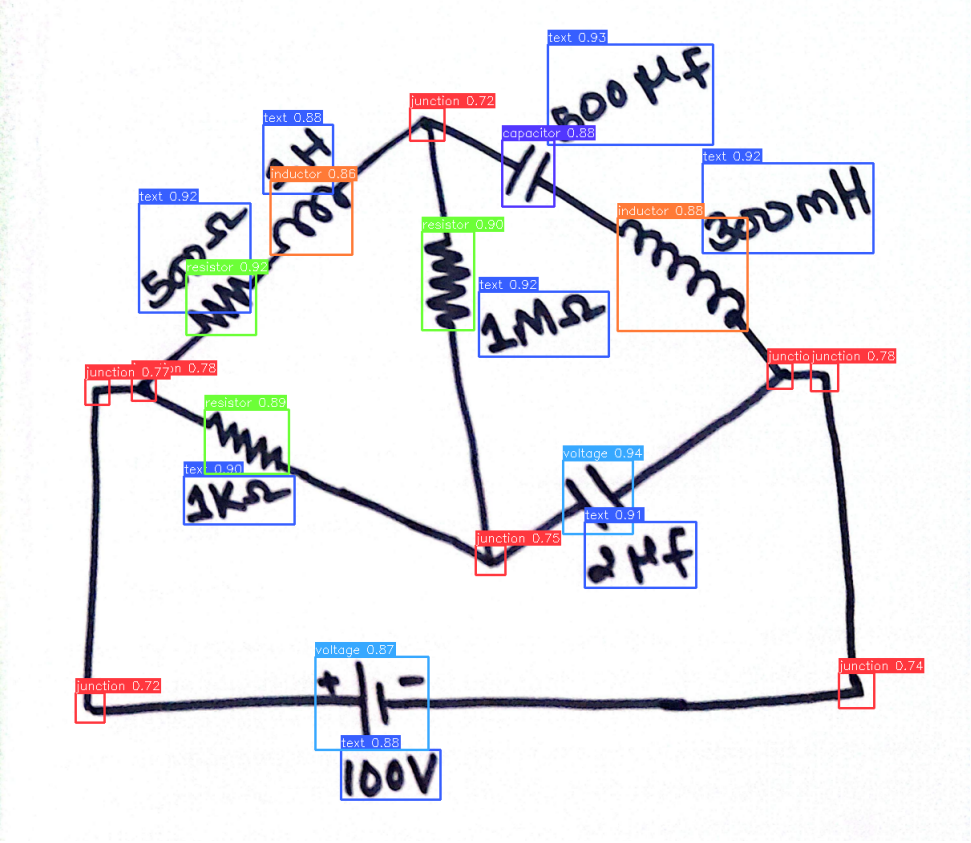


Figure 6-12 Retrained YOLOv7 Result (i)

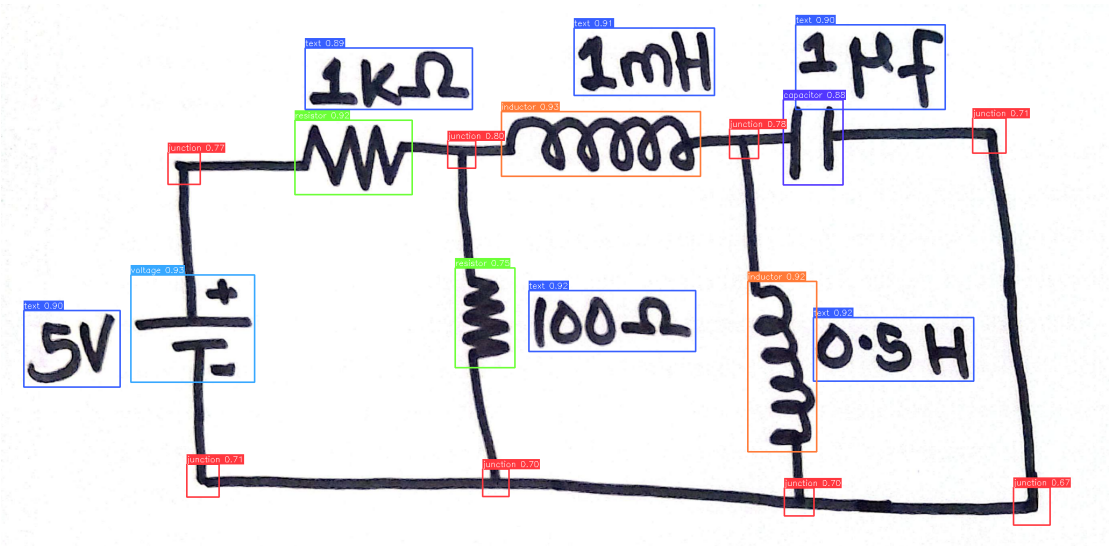


Figure 6-13 Retrained YOLOv7 Result (ii)

Figure 6-12 and Figure 6-13 show the results of the retrained YOLOv7 on two random test images. The model successfully detects and classifies all components in both images

with high confidence scores. These results visually confirm the quantitative metrics and demonstrate the model's effectiveness in real-world scenarios.

6.5 Custom OCR Results and Analysis

6.5.1 Custom OCR Training Loss Curve

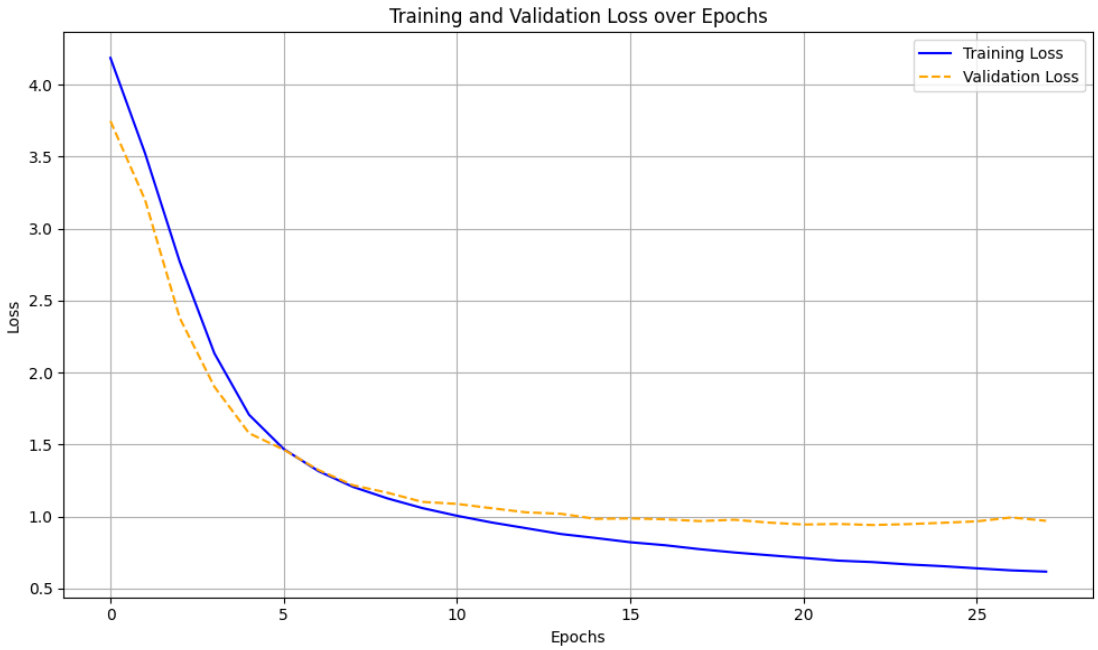


Figure 6-14 Training Loss vs Epochs (Custom OCR)

Figure 6-14 shows that the training loss decreases consistently throughout the 27 epochs, indicating the model is effectively memorizing the training data. There seems to be the slightest uptick in the validation loss, though, at 26 epoch indicating a hint of potential overfitting. The training was stopped at 27 epochs due to early stopping criteria being met.

6.5.2 Custom OCR Performance Analysis

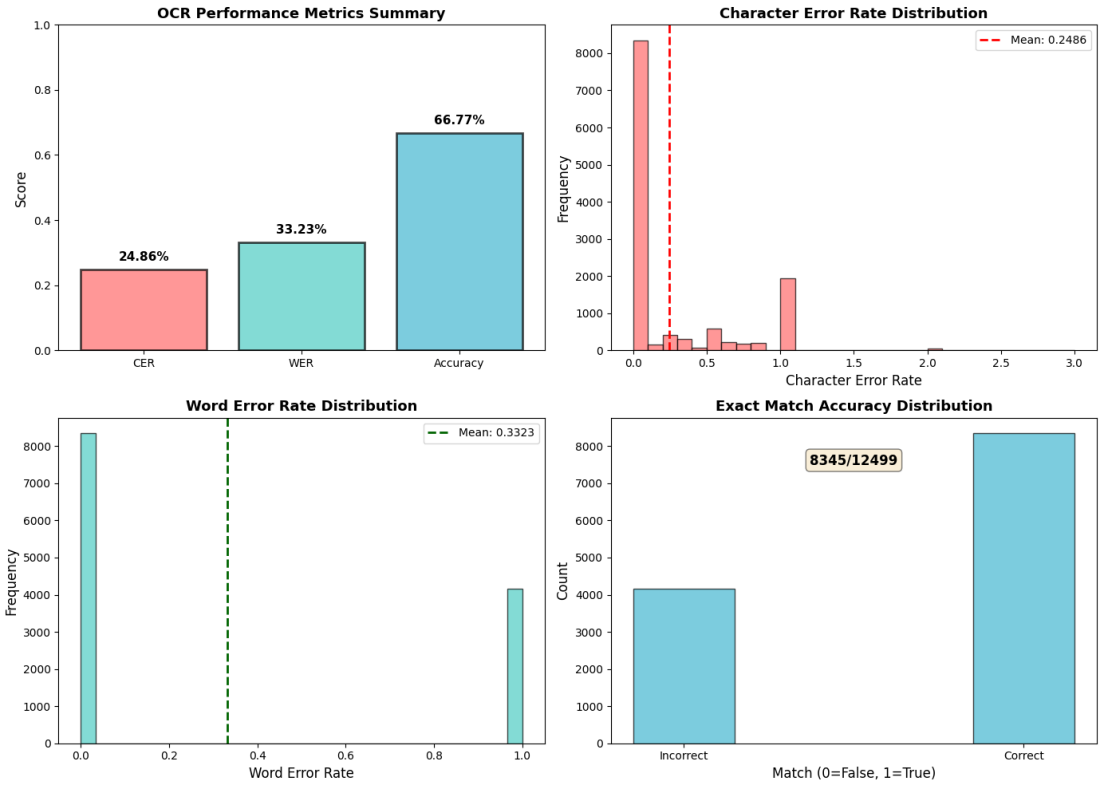


Figure 6-15 Custom OCR Performance Metrics and their Distributions

Figure 6-15 is a combination of images showing the performance metrics for the custom OCR model. The accuracy is 66.77%. Likewise, the CER of approximately 0.25 means that on average, 1 out of every 4 characters is predicted incorrectly. However, the histogram shows a notable bump at 1.0 (complete failure) confirming the presence of "garbage" inputs which can be fixed by the use of data cleaning techniques. WER is shown to be 0.33. Its distribution is inherently binary because the model either gets the full component value right (WER being 0) or gets it wrong (WER = 1).

The performance metrics indicate that while the custom OCR model has learned to recognize some component values, there is significant room for improvement. This sets the stage for TrOCR which is expected to have much better performance due to its pre-trained weights and more advanced architecture.

6.6 Comparative Analysis of OCR Models

Two distinct OCR architectures were evaluated for extracting component values (e.g., “10kΩ”, “100μF”) from detected text regions: the custom CRNN-based model trained from scratch, and a fine-tuned TrOCR-small model based on the Vision Transformer architecture as mentioned in previous sections.

6.6.1 Quantitative Comparison

Table 6-5 Performance comparison of OCR models

Metric	Custom CRNN	Fine-tuned TrOCR
Accuracy (%)	66.77	**84.50**
Character Error Rate (CER)	0.25	**0.12**
WER	0.33	**0.18**
Training Epochs	27	3
Batch Inference	Sequential	Parallel (GPU)
Precision (float)	float32	float16

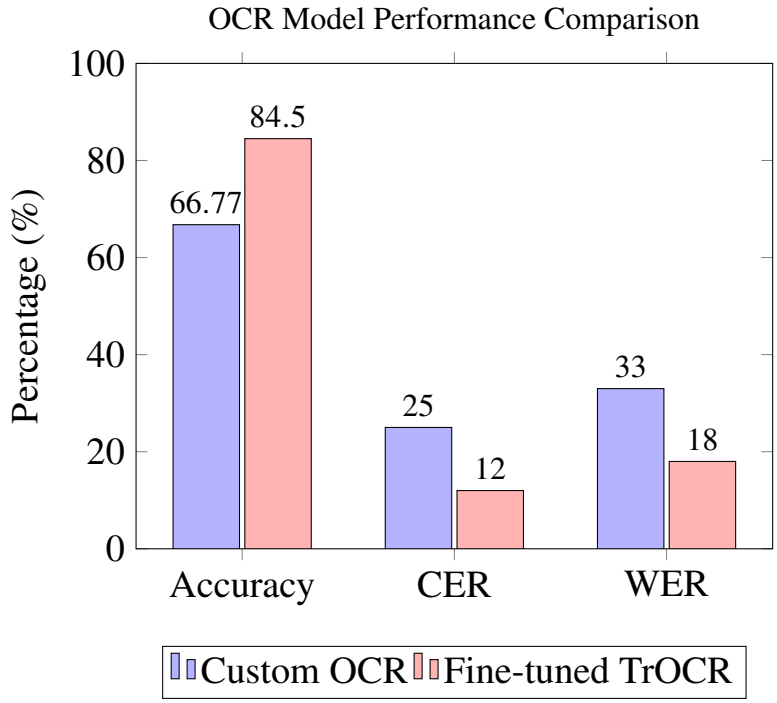


Figure 6-16 Performance metrics comparison, in percentage

Table 6-5 summarizes the performance of both OCR models, and Figure 6-16 provides a visual comparison. The fine-tuned TrOCR model outperforms the custom CRNN across all metrics despite requiring only 3 training epochs compared to 27 (tested on the test split of the CGHD dataset). The CER reduction from 0.25 to 0.12 indicates that TrOCR makes half as many character-level errors, directly improving the accuracy of extracted component values. It is noted that the custom CRNN's lower accuracy (66.77%) was partly attributed to data corruption during preprocessing. Regardless, the architectural advantages of the Transformer-based approach make TrOCR the superior choice for this pipeline. However, the inference speed of TrOCR has found to be considerably slower than the custom OCR model. This implies there is inherently a trade-off between accuracy and inference speed when choosing between the custom OCR and TrOCR

models for this project. Keeping this in mind, both models are retained in the codebase and can be switched based on user preference.

6.7 OCR Demonstration

Two randomly picked images from the dataset are shown below after passing through the detection (YOLOv7) and recognition (OCR) models.

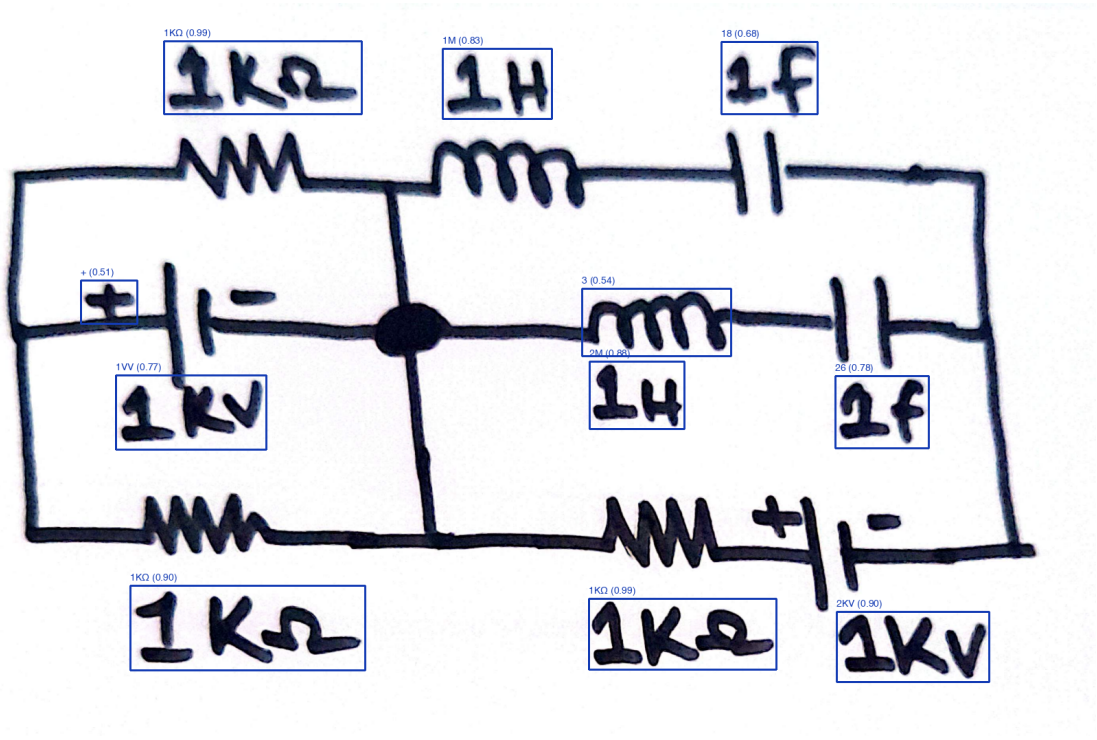


Figure 6-17 Text Detection and Recognition using Custom OCR(i)

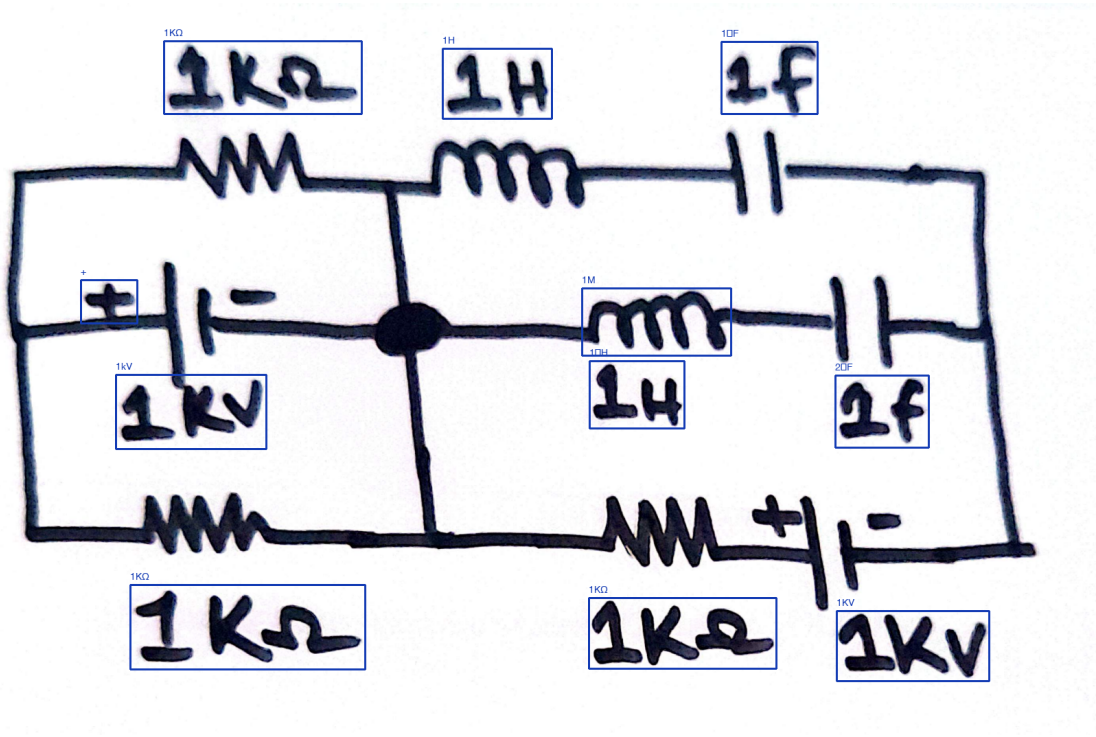


Figure 6-18 Text Detection and Recognition using TrOCR(i)

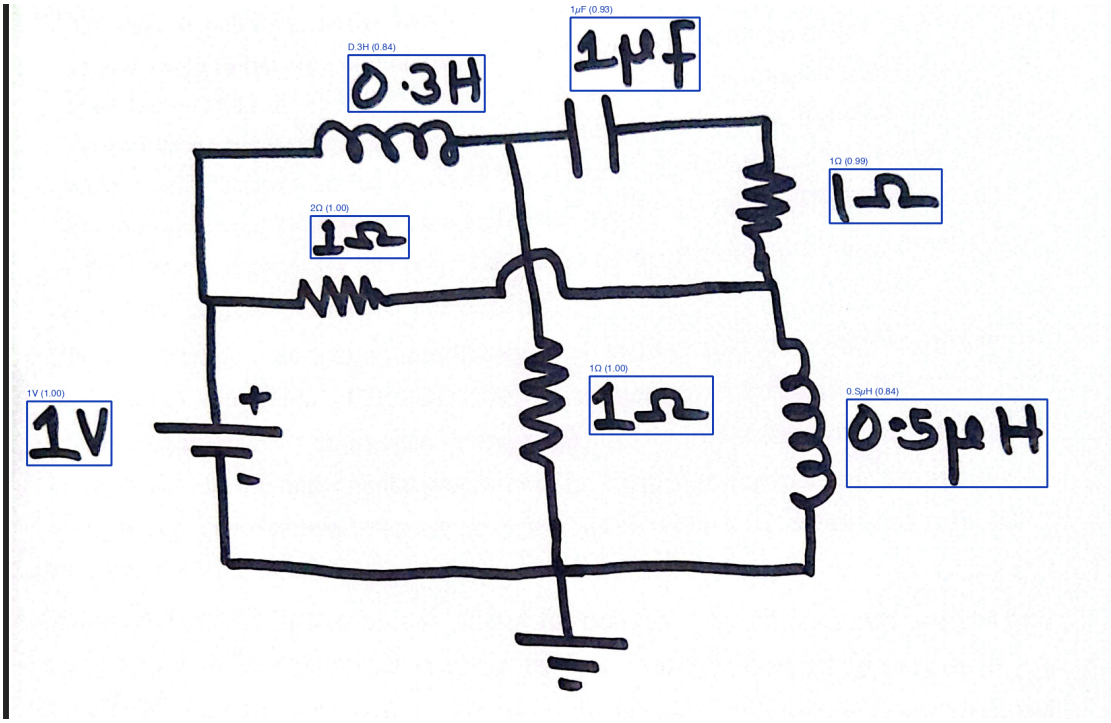


Figure 6-19 Text Detection and Recognition using Custom OCR(ii)

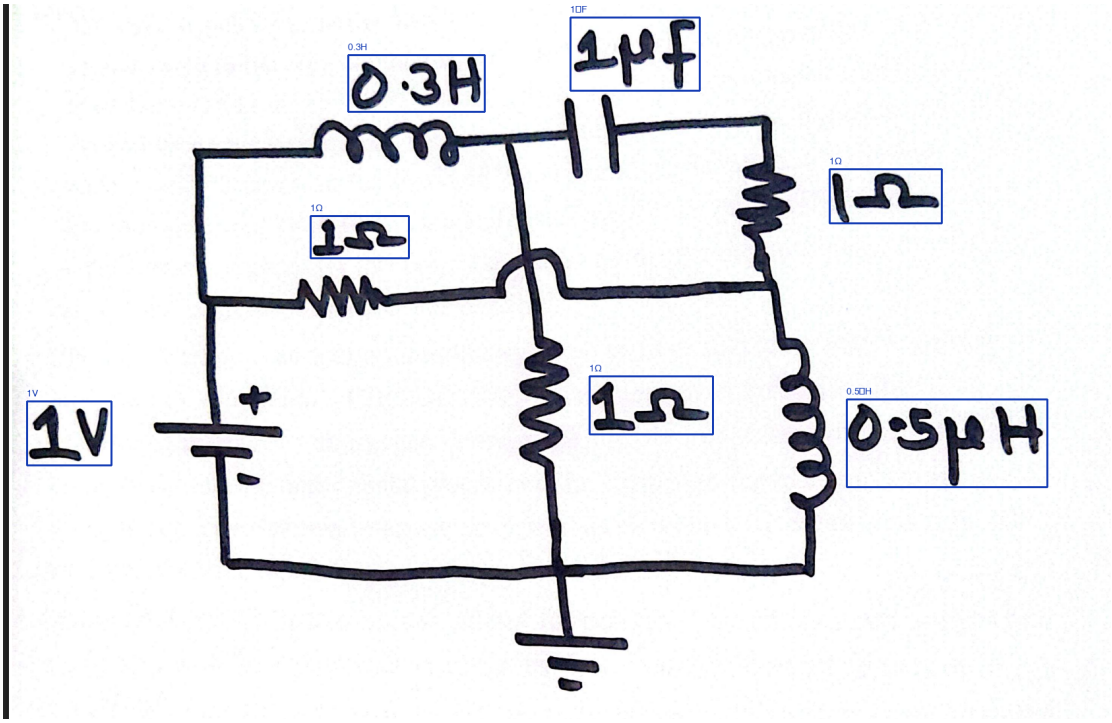


Figure 6-20 Text Detection and Recognition using TrOCR(ii)

Figure 6-17 and Figure 6-19 show the results of the custom OCR model on two random test images. The model successfully detects text regions and extracts component values, albeit with many errors. In contrast, Figure 6-18 and Figure 6-20 demonstrate the superiority in performance of the fine-tuned TrOCR model, which correctly recognizes more component values.

6.8 Junction-Based Wire Detection

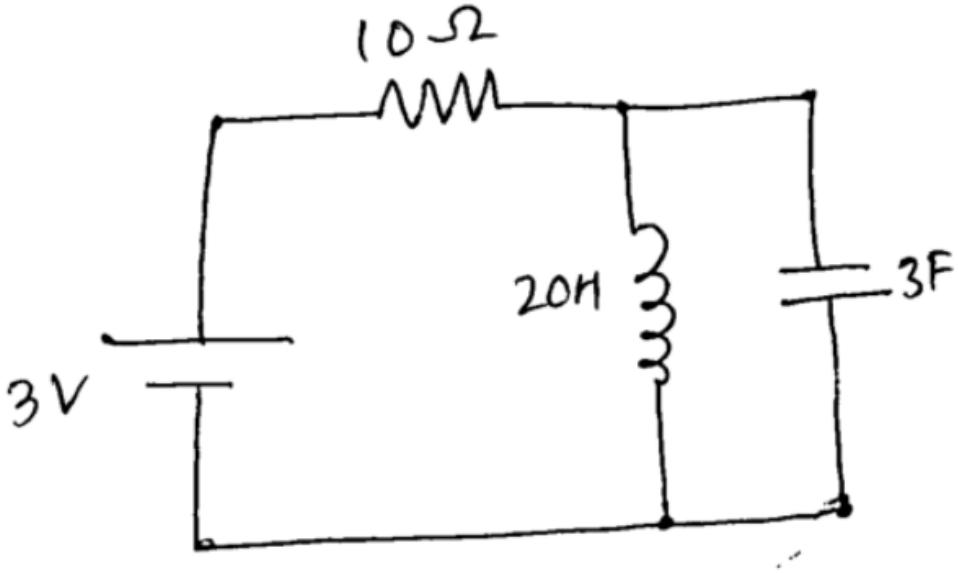


Figure 6-21 Otsu's Global Thresholding

What follows are the experimental outcomes from running the junction-based wire detection pipeline on circuit diagram images. A high-resolution scanned diagram serves as input, with its pixel dimensions (width and height) recorded upfront so that spatial scale stays consistent across subsequent stages. The raw image is first converted to grayscale to sharpen structural features and tamp down background noise. Otsu's global thresholding then kicks in, computing an optimal cutoff that cleanly separates foreground circuit elements from everything behind them. As Figure 6-21 illustrates, the resulting binary image retains wires and junctions while discarding illumination artifacts.

From the binary image, morphological skeletonization thins every line down to a single pixel in width, preserving connectivity throughout. This simplification makes it far easier to trace wire paths and locate junctions. Bounding boxes from the YOLOv7 object detection stage are then superimposed on the skeleton to pinpoint where wires cross component boundaries. Each detected endpoint gets marked on the skeleton, and its spatial coordinates are logged for the stages that follow.

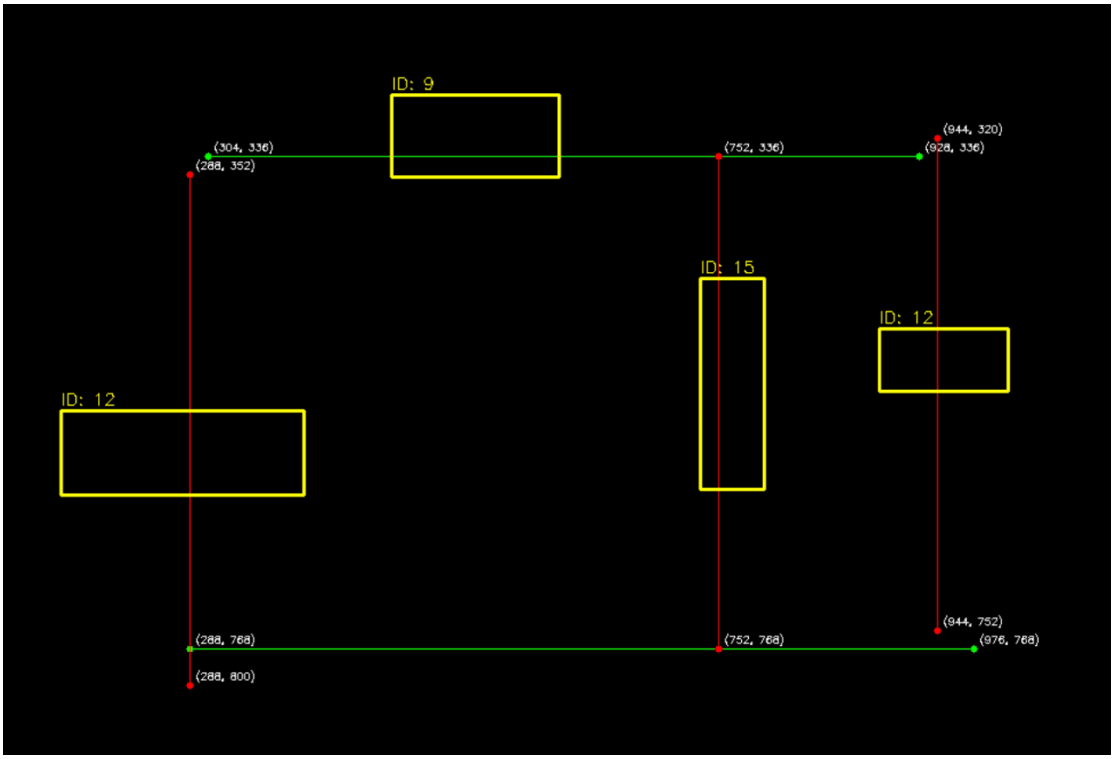


Figure 6-22 Bounding Box Overlay

Figure 6-22 shows the skeleton with its bounding box overlay. Where wires cross the edges of component bounding boxes, intersection points are calculated to establish electrical connectivity. The total count of such intersections is reported alongside a directional split between horizontal and vertical wire crossings at specific component edges. These crossing points underpin the reconstruction of circuit connectivity graphs and the generation of CircuitJS-compatible text files.

That said, this junction-based approach carries a serious limitation that the diagonal lines break it. The bounding box intersection logic assumes wires will meet component boundaries along strictly horizontal or vertical axes. When a wire runs at an angle, it may miss the box edges entirely or hit them unpredictably, which leaves connections undetected and the resulting graph incomplete. Because of this weakness, the method was set aside in favor of a path-finding approach capable of handling wires at arbitrary orientations.

6.9 Line Detection via Path Finding Visualization

6.9.1 Skeletonized Image with Detected Endpoints

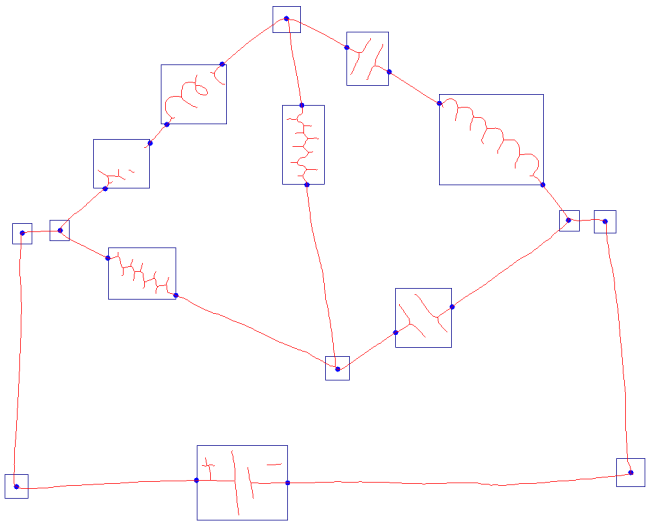


Figure 6-23 Skeletonized image with detected endpoints

Figure 6-23 overlays every detected endpoint on the skeletonized image. The skeleton itself is the one-pixel-wide remnant of the circuit lines after morphological processing. Blue dots mark the endpoints, which were located through bounding box edge intersections for line segments and center-snapping for junctions. Blue rectangles correspond to the original YOLO bounding boxes that guided endpoint localization. The visualization verifies that the detection algorithm is reliably picking up connection points along object boundaries.

6.9.2 Adjacency Graph Visualization

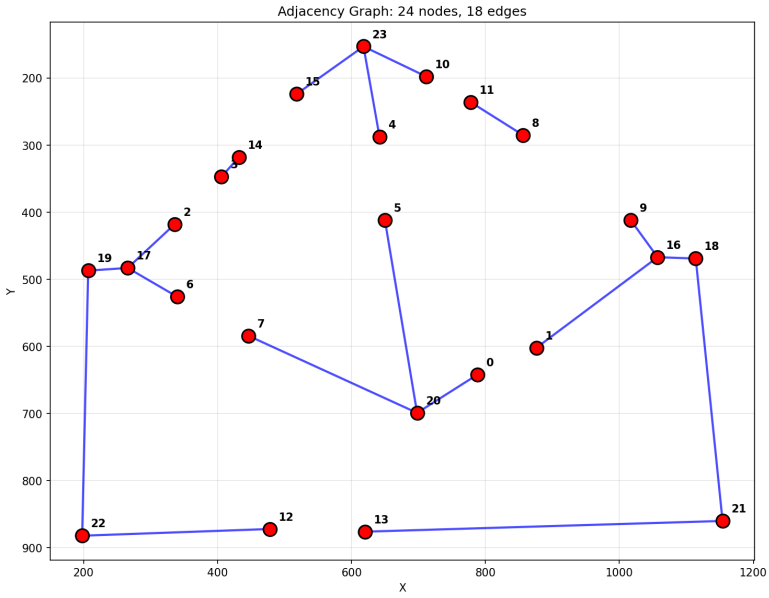


Figure 6-24 Adjacency graph constructed using Multi-Head BFS algorithm

Figure 6-24 shows the adjacency graph that the Multi-Head BFS algorithm produced. Every node maps to one of the endpoints detected in the prior stage, and the blue lines between nodes are graph edges that reflect skeleton-level connectivity. Self-edges, which would falsely link two endpoints belonging to the same detected object, have been pruned away so that only genuine inter-component connections survive. The resulting graph feeds directly into circuit topology analysis and netlist extraction further downstream.

6.10 Combined Overlay of YOLOv7, TrOCR, and Line Detection Results

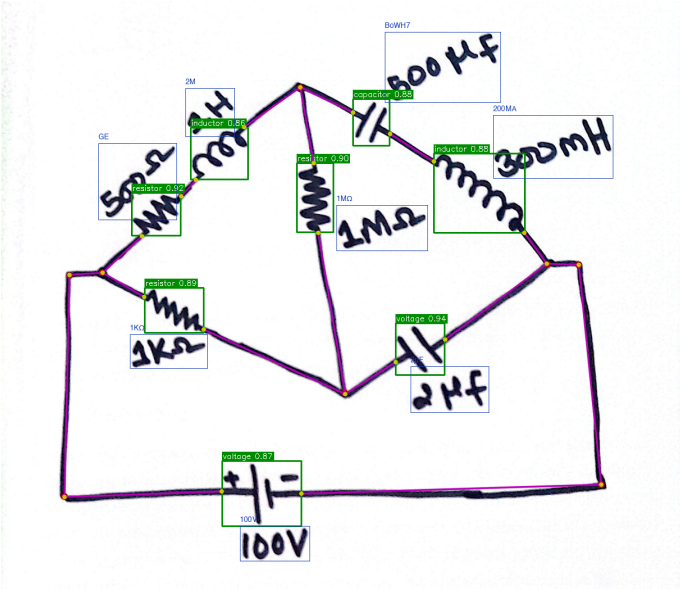


Figure 6-25 Combined overlay of YOLOv7, TrOCR, and line detection results

Figure 6-25 layers the outputs of object detection (YOLOv7), text recognition (TrOCR), and line detection (adjacency graph) onto a single image. The original circuit diagram sits in the background. Green rectangles are the YOLOv7 bounding boxes, blue annotations are the component values pulled out by TrOCR, and red lines trace the detected wiring between components. Seeing all of these stacked together confirms that the pipeline correctly locates components, reads off their values, and rebuilds the wiring layout.

6.11 Frontend Visualization

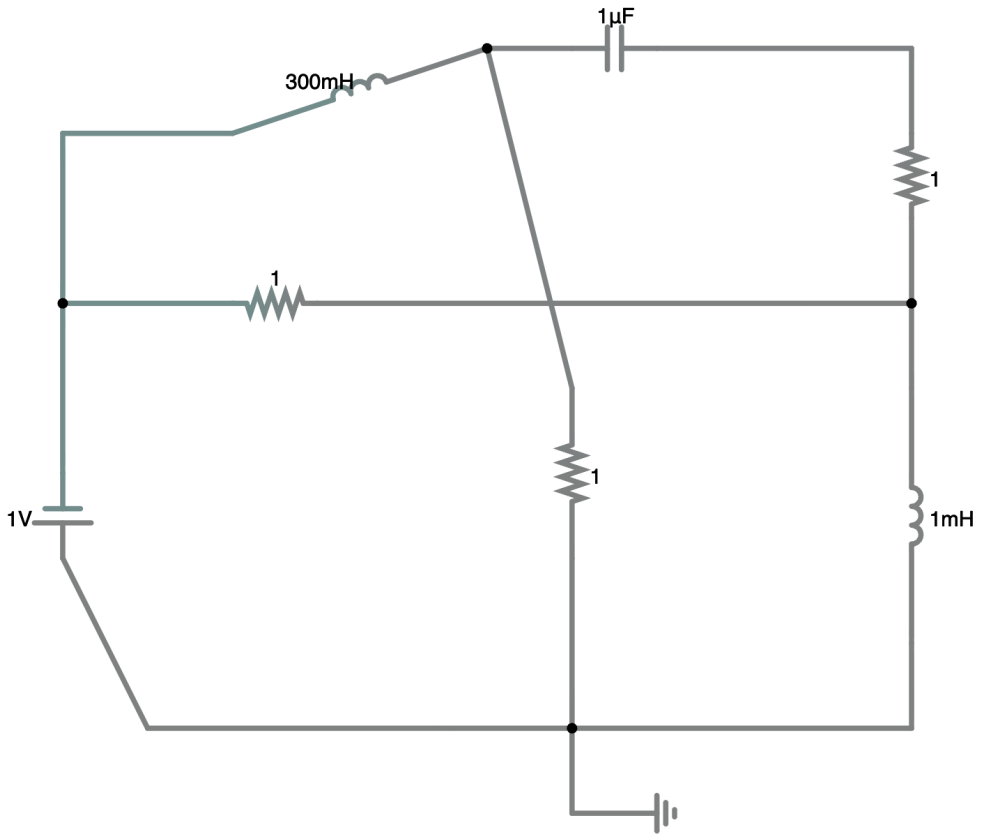


Figure 6-26 Digitally Drawn Circuit (Color Inverted)

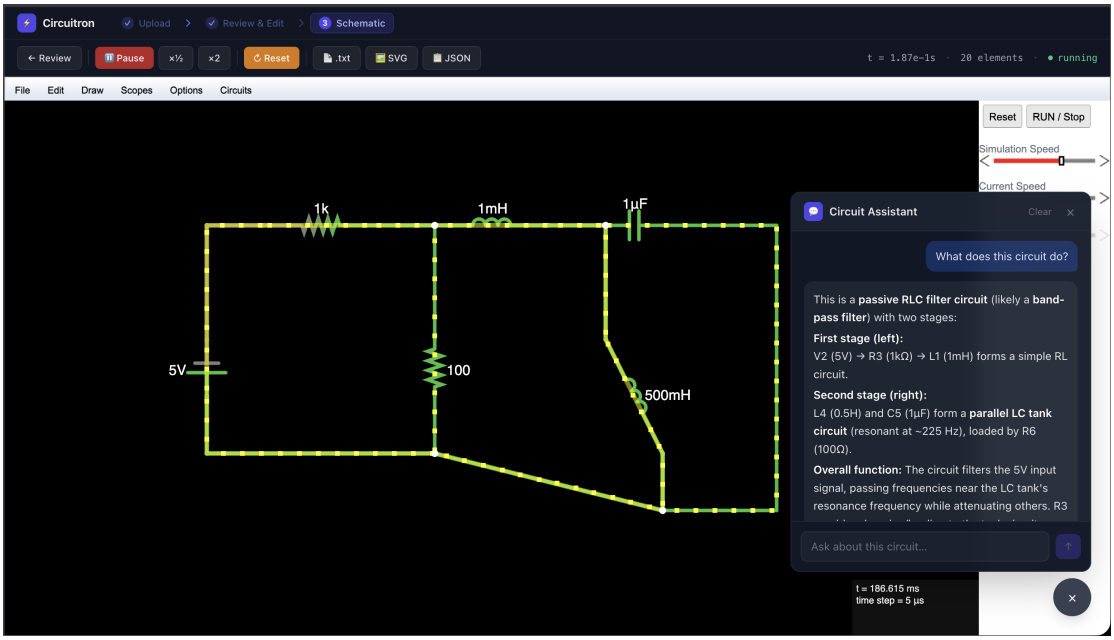


Figure 6-27 Fullscreen Capture of the Frontend

Figure 6-26 shows an editable, digital circuit auto-generated by the system. This is the output before any manual intervention. Figure 6-27 shows the fullscreen capture of the frontend with a sample circuit as well as the Q&A section using DeepSeek v3.1.

Circuitron's frontend gives users an interactive workspace for designing and simulating circuits. The schematic editor supports drag-and-drop placement on a snappable grid with live wire routing. Passive elements, active devices, and sources each expose a properties panel where electrical parameters and positioning can be tuned with precision.

Transient analysis controls are built in. Users set their time parameters, fire off a simulation, and watch the results appear as dynamically rendered voltage-time plots. Measurement probes let users monitor signals in real time while the system automatically computes peak-to-peak swing and Root Mean Square (RMS) values alongside maxima and minima. Simulation data and plots can also be exported in standard formats for offline review or inclusion in documentation.

7 FUTURE ENHANCEMENTS

The CIRCUITRON system, while fully functional in its current state, has numerous opportunities for future enhancements. One of the objectives of any research-based project is to encourage the scientific community to conduct further studies and experiment on the working principle of the project to either verify the methodology used, or potentially come up with alternate techniques to solve the same probelem. In any case, following are the likely future enhancements that can be implemented in this project:

7.1 Improvement in the OCR accuracy

The system uses two OCR models, at the moment, however the accuracy of both of them (highest of them being around 85%) has room for improvement. Also, the issue with TrOCR (which is the better of the two) is that it is too slow on account of its large size. The custom OCR pipeline is much faster, but has lower accuracy. Future work can be done to either improve the accuracy of the custom OCR model though rigorous training on different datasets, or to somehow improve TrOCR's speed.

7.2 Increment in the number of supported components

The system currently supports a limited number of components (only 12 after consolidation, minus crossover, junction, and text classes). Pre-consolidation, the system supported 59 components, but the mAP was too low for satisfactory object detection. These 12 components are the most commonly used ones for basic circuit design and simulation, however, there are many more components that can be added to the system to make it more versatile and useful for a wider range of applications. Future work can be done to increase the number of supported components, which would require training the object detection model on a larger and more diverse dataset.

7.3 Authentication and Database Functionalities

The system, in its current form, doesn't allow the user to save their work or to access it later. Moreover, there is no authentication mechanism in place to allow users to have personalized accounts and potentially collaborate on the same projects. This also involves hosting the system online. Future enhancements can include the integration of a database to store user data and circuit designs, as well as offer a library of pre-designed circuits for beginners to play around with.

7.4 Improvement in the frontened and general user experience

While the system is capable of handling diagonal wires and components, when said components are rendered in the frontend, they aren't rendered in the same diagonal

manner. This requires the user to step in and manually adjust the orientation of the component. The issue of scale, between the wires and the components also persists, ocassionally requiring manual resizing. Moreover, the frontend of the system involves a lot of user interactions from dragging and resizing, to general drawing and editing of the schematic, which can be made smoother in the future.

8 CONCLUSION

The objectives of recognizing a hand-drawn circuit diagram, transforming it into an editable schematic having simulation capabilities, and analyzing said circuit with LLM-powered Q&A have been successfully achieved. Functional pipelines for each aspect of the project have been designed and implemented effectively. For component detection, YOLOv7 has achieved a mAP@0.5 of 95% and for text recognition, TrOCR has achieved accuracy of 84.5%. Through the use of junction-based geometric inference and collinear merging, the wire detection algorithm extracts circuit connectivity. Overall, all these elements work hand-in-hand to extract components with their values and connections, effectively digitizing a circuit. Hence, the project has successfully addressed a significant gap in educational circuit analysis by making circuit simulation easier than ever.

Beyond the ML inference side of things, the project also pulls off a working full-stack deployment: a React.js frontend talks to a FastAPI backend, which in turn hooks into CircuitJS for running simulations. Keeping these layers modular makes the codebase straightforward to maintain and opens up room for future additions.

Building Circuitron doubled as a research exercise for the authors. Comparing several YOLO generations for component detection, weighing different OCR architectures for text recognition, and wrestling with multiple strategies for resolving wire connectivity pushed the project toward the strongest configuration it could reach. Along the way, it gave the authors hands-on familiarity with a broad swathe of current scientific methods. Circuitron is, at its core, an attempt to lower the barrier to circuit education by shrinking the gap between sketching an idea and actually analyzing it. The design pattern behind it is both replicable and improvable, which makes it useful to the wider electronics education community. Where conventional simulation tools tend to be either costly or intimidating, this system has the potential to keep students more engaged while easing the cognitive overhead of learning circuit analysis.

9 APPENDICES

Appendix A: Project Budget

Table 9-1 Project Budget Breakdown

Category	**Details**	**Estimated Cost (NPR)**
Computation Requirements	Google Colab Pro ($9.99 per month), NVIDIA T4 GPU ($0.35 per hour), Lightning AI Credits	24,000
IEEE Xplore Membership	Annual subscription for accessing research articles, journals	2,160
Miscellaneous Costs	Report printing, binding, API, etc.	15,000
Total		**41,160**

Appendix B: Gantt Chart

Table 9-2 Project Timeline (Gantt Chart)

PROCESS	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar
Research											
Implementation											
Testing											
Documentation											
Final Submission											

REFERENCES

[1] R. R. Reddy and M. R. Panicker, "Hand-drawn electrical circuit recognition using object detection and node recognition," *arXiv preprint arXiv:2106.11559*, Jan 2021, available: https://arxiv.org/abs/2106.11559.

[2] A. E. Sertdemir, M. Besenk, T. Dalyan, Y. D. Gokdel, and E. Afacan, "From image to simulation: An ANN-based automatic circuit netlist generator (Img2Sim)," in *Proc. IEEE Int. Conf. on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD)*, 2022, pp. 1–4.

[3] Y.-L. Chen, Y.-R. Hong, Y.-T. Sung, and K.-E. Chang, "Efficacy of simulation-based learning of electronics using visualization and manipulation," *Educational Technology & Society*, vol. 14, no. 2, pp. 269–277, 2011.

[4] B. Edwards and V. Chandran, "Machine recognition of hand-drawn circuit diagrams," in *Proc. IEEE Int. Conf. on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 6, Nov 2000, pp. 3618–3621.

[5] J.-Y. Ramel, G. Boissier, and H. Emptoz, "A structured approach for the recognition of hand-drawn technical documents," in *Proc. Int. Workshop on Graphics Recognition (GREC)*. Springer, 2000, pp. 141–152.

[6] S. Belongie, J. Malik, and J. Puzicha, "Shape matching and object recognition using shape contexts," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 4, pp. 509–522, 2002.

[7] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 580–587.

[8] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015, pp. 1440–1448.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 91–99.

[10] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2961–2969.

[11] J. Bayer, A. K. Roy, and A. Dengel, "Instance segmentation based graph extraction for handwritten circuit diagram images," *arXiv preprint arXiv:2301.03155*, Jan 2023, available: https://arxiv.org/abs/2301.03155.

[12] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. European Conf. Comput. Vis. (ECCV)*. Springer, 2016, pp. 21–37.

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[14] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 6517–6525.

[15] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[16] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[17] G. Jocher, "YOLOv5," https://github.com/ultralytics/yolov5, 2020, accessed: 2025-01-15.

[18] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 7464–7475.

[19] M. F. Adnan, N. Ahmed, A. Shafiullah, M. S. Rahman, and G. Sarowar, "A two-stage CNN-based hand-drawn electrical and electronic circuit component recognition system," *Soft Computing*, vol. 33, pp. 13 367–13 390, 2021.

[20] M. S. Abhinav, V. L. Shrimanth, P. N. Ganesh, P. V. Nishanth, and K. Bhargavi, "Electric circuit diagram detection, recognition and simulation," *International Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 9, pp. 3221–3227, Sep 2020.

[21] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, 2017.

[22] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 369–376.

[23] J. Baek *et al.*, "What is wrong with scene text recognition model comparisons? dataset and model analysis," *arXiv preprint arXiv:1904.01906*, Apr 2019, available: https://arxiv.org/abs/1904.01906.

[24] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," *arXiv preprint arXiv:2109.10282*, 2021, available: https://arxiv.org/abs/2109.10282.

[25] R. O. Duda and P. E. Hart, "Use of the hough transformation to detect lines and curves in pictures," *Commun. ACM*, vol. 15, no. 1, pp. 11–15, 1972.

[26] J. Matas, C. Galambos, and J. Kittler, "Robust detection of lines using the progressive probabilistic hough transform," *Computer Vision and Image Understanding*, vol. 78, no. 1, pp. 119–137, 2000.

[27] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Commun. ACM*, vol. 27, no. 3, pp. 236–239, 1984.

[28] D. Hemker, J. Maalouly, H. Mathis, R. Klos, and E. Ravanan, "From schematics to netlists–electrical circuit analysis using deep-learning methods," *Advances in Radio Science*, vol. 22, pp. 61–75, Nov 2024.

[29] C. R. Kelly and J. M. Cole, "Digitizing images of electrical-circuit schematics," *APL Machine Learning*, vol. 2, no. 1, p. 016109, Jan 2024.

[30] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[31] Y. Yamakaji, H. Shouno, and K. Fukushima, "Circuit2graph: Circuits with graph neural networks," *IEEE Access*, vol. 11, pp. 123 456–123 467, 2023.

[32] KiCad, "SPICE simulation," https://www.kicad.org/discover/spice/, 2024, accessed: 2025-01-15.

[33] L. W. Nagel and D. O. Pederson, "SPICE (simulation program with integrated circuit emphasis)," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 1973, univ. California, Berkeley, ERL Memo ERL-M382.

[34] P. Falstad, "Circuit simulator," https://www.falstad.com/circuit/circuitjs.html, 2026, accessed: Mar. 22, 2026.

[35] M. Li *et al.*, "CircuitVQA: A visual question answering dataset for electrical circuit images," *arXiv preprint arXiv:2407.03056*, 2024.

[36] D. Vungarala, S. Alam, A. Ghosh, and S. Angizi, "SPICEPilot: Navigating SPICE code generation and simulation with AI guidance," *arXiv preprint arXiv:2410.20553*, Oct 2024, available: https://arxiv.org/abs/2410.20553.

[37] J. Solawetz, “What is YOLOv7? A complete guide,” Roboflow Blog, https://blog.roboflow.com/yolov7-breakdown/, Jan 2024, accessed: Mar. 23, 2026.